Catalog

1. DBMS Introduction 1 1
2. DBMS Introduction 2 15
3. DBMS ER - Diagrams 38
4. DBMS EER - Diagrams 63
5. DBMS Keys 75
6. DBMS Relational Algebra 91
7. DBMS Functional Dependency 136
8. DBMS Armstrong Axioms (Inference Rules) 147
9. DBMS Normalization Part 1 165
10. DBMS Normalization Part 2 177
11. DBMS Normalization Part 3 192
12. DBMS Transaction Processing 202
13. DBMS Concurrency Control Techniques 217
14. DBMS Log Based Recovery 229
15. DBMS Deadlock 237
16. DBMS Serializability 247
17. DBMS Distributed Database 260
18. DBMS File Organization 270

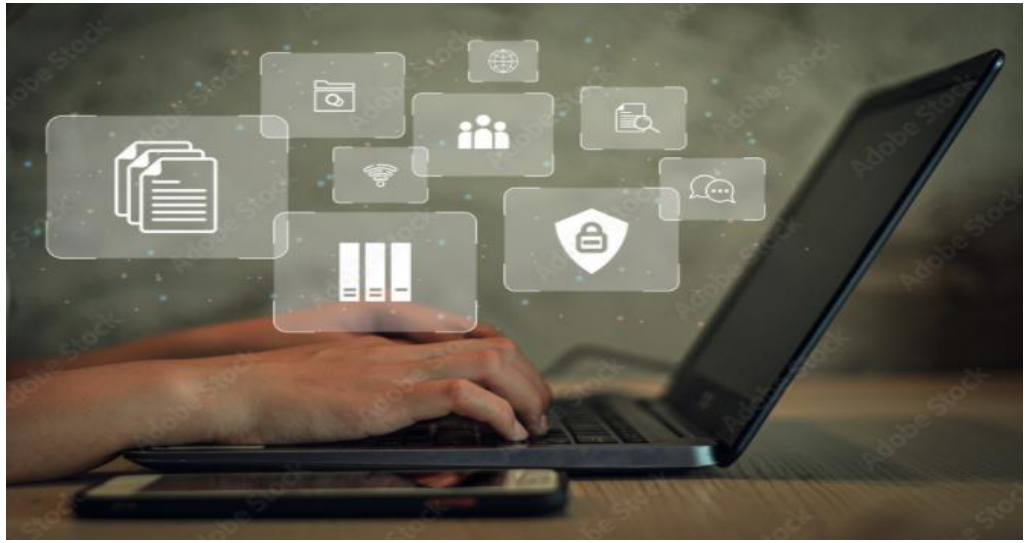
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC-1003)

Topic: Introduction to DBMS



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Syllabus

Credits: 03 **L-T-P-J: 3-0-0-0**

Module No.	Content	Teaching Hours
I	Introduction: An Overview of Database Management System, Database System Vs File System, Database System Concept and Architecture, Data Model Schema and Instances, Data Independence, Database Language and Interfaces (DDL, DML, DCL), Database Development Life Cycle (DDLC) with Case Studies. Data Modeling Using the Entity-Relationship Model: ER Model Concepts, Notation for ER Diagram, Mapping Constraints, Keys, Specialization, Generalization, Aggregation, Reduction of an ER Diagram to Tables, Extended ER Model. Relational Data Model and Language: Relational Data Model Concepts, Integrity Constraints, Entity Integrity, Referential Integrity, Keys Constraints, Domain Constraints, Relational Algebra Database Design & Normalization I: Functional Dependencies, Primary Key, Foreign Key, Candidate Key, Super Key, Normal Forms, First, Second, Third Normal Forms, BCNF, Non-Redundant Cover, Canonical Cover	20

Syllabus

II	**Database Design & Normalization II:** 4th Normal Form, 5th Normal Form, Lossless Join Decompositions, MVD and JDs, Inclusion Dependence. **File Organization:** Indexing, Structure of Index files and Types, Dense and Sparse Indexing **Transaction Processing Concept:** Transaction System, Testing of Serializability, Serializability of Schedules, Conflict & View Serializable Schedule, Recoverability, Recovery from Transaction Failures, Log Based Recovery, Deadlock Handling. **Concurrency Control Techniques:** Concurrency Control, Locking Techniques for Concurrency Control, 2PL, Time Stamping Protocols for Concurrency Control, Validation Based Protocol. **Distributed Database:** Introduction of Distributed Database, Data Fragmentation and Replication.	20

Suggested Books

Text Books:

- Elmasri and Navathe, “Fundamentals of Database Systems”, 6th Edition, Addision Wesley, 2010.
- Sadalage, P. & Flower, “NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Presistence”, Pearson Education, 2012.

Reference Books:

- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- Korth, Silbertz and Sudarshan, “Database Concepts”, 5th Edition, TMH, 1998.
- Redmond, E. & Wilson, “Seven Databases in Seven Weeks: A Guide to Modern Databases and the NoSQL Movement”, 1st Edition.

Database Management System (DBMS)

- A Database Management System (DBMS) is a software designed to store, retrieve, define, and manage data in a database.

- DBMS software primarily functions as an interface between the end user and the database, simultaneously managing the data, and the database schema in order to facilitate the organization and manipulation of data.

- A database management system functions through the use of system commands, first receiving instructions, then instructing the system accordingly, either to retrieve data, modify data, or load existing data from the system.

Difference Between Data & Information in DBMS

- Data is raw, unprocessed, unorganized facts that are seemingly random and do not yet carry any significance or meaning.

- Information refers to data that has been organized, interpreted, and contextualized by a human or machine so that it possess relevance and purpose.

- Information is filtered data that has been made systematic and useful, and is considered to be more reliable and valuable to researchers as proper analysis and refinement has been conducted.

File System Vs DBMS

File System:

- The file system is basically a way of arranging the files in a storage medium like a hard disk.
- The file system organizes the files and helps in the retrieval of files when they are required.
- File systems consist of different files which are grouped into directories.
- The directories further contain other folders and files.
- The file system performs basic operations like management, file naming, giving access rules, etc.

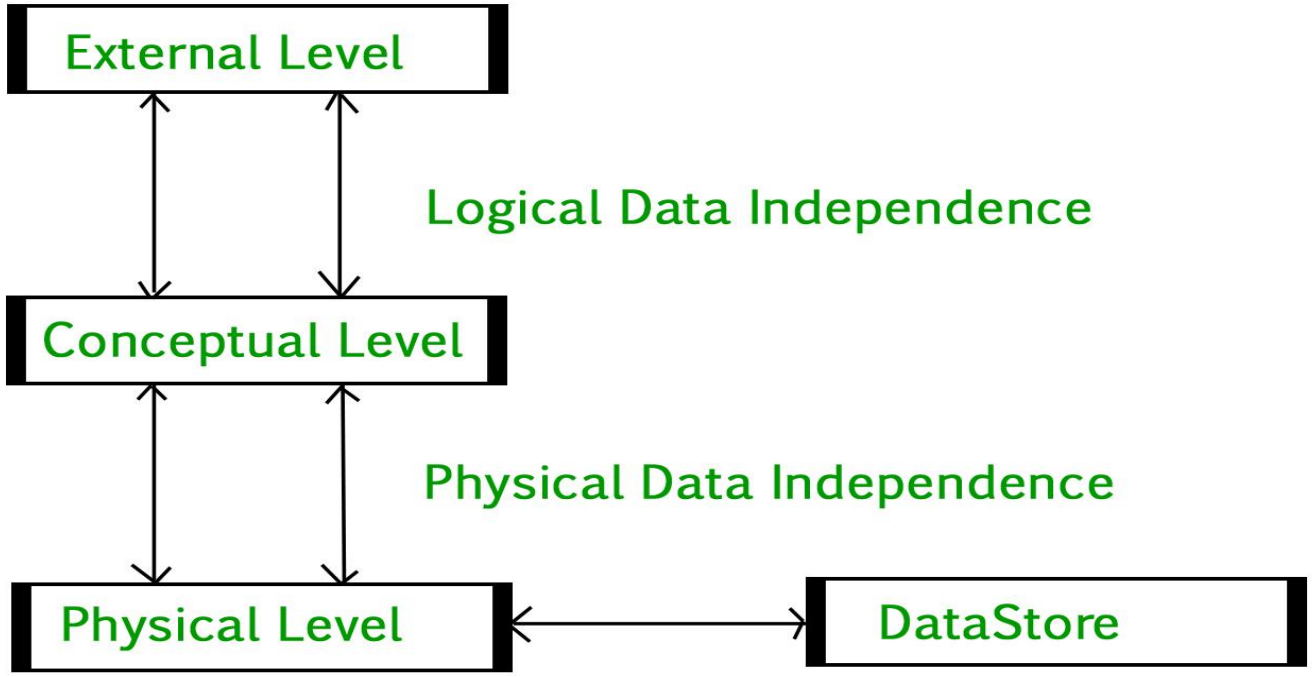
File System Vs DBMS

DBMS:

- Database Management System is basically software that manages the collection of related data.
- It is used for storing data and retrieving the data effectively when it is needed.
- It also provides proper security measures for protecting the data from unauthorized access.
- In Database Management System the data can be fetched by queries and relational algebra.
- It also provides mechanisms for data recovery and data backup.

Introduction of 3-Tier Architecture in DBMS

- DBMS 3-tier architecture divides the complete system into three inter-related but independent modules as shown below:



Introduction of 3-Tier Architecture in DBMS

Physical Level:

- At the physical level, the information about the location of database objects in the data store is kept.
- Various users of DBMS are unaware of the locations of these objects.
- In simple term, physical level of a database describes how the data is being stored in secondary storage devices like disks and tapes.

Introduction of 3-Tier Architecture in DBMS



Conceptual Level:

- At conceptual level, data is represented in the form of various database tables.
- For Example, STUDENT database may contain STUDENT and COURSE tables which will be visible to users but users are unaware of their storage.
- Also referred as logical schema, it describes what kind of data is to be stored in the database.

Introduction of 3-Tier Architecture in DBMS

External Level:

- An external level specifies a view of the data in terms of conceptual level tables.
- Each external level view is used to cater to the needs of a particular category of users.
- For Example, FACULTY of a university is interested in looking course details of students, STUDENTS are interested in looking at all details related to academics, accounts, courses and hostel details as well.
- So, different views can be generated for different users.
- The main focus of external level is data abstraction.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 5th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 6th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you

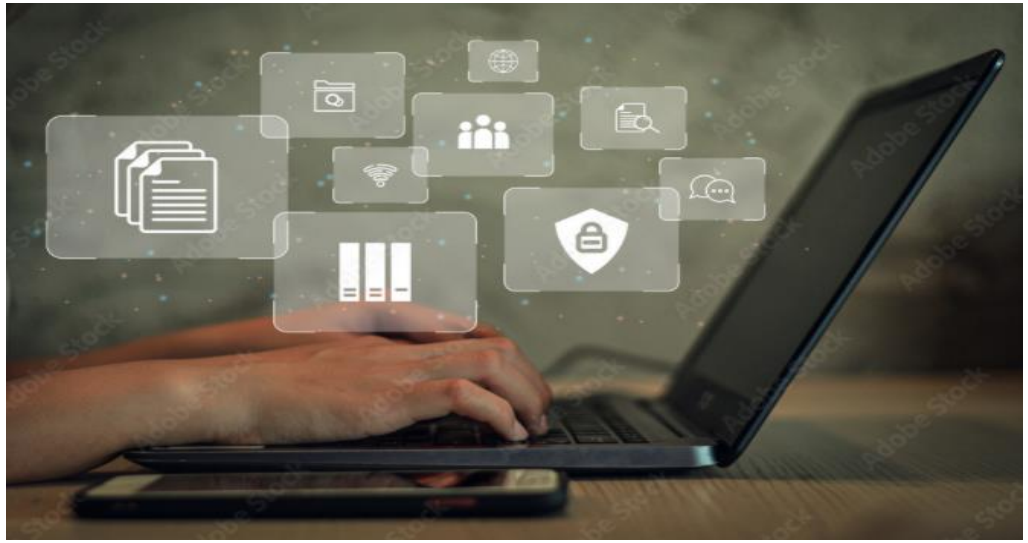
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC-1003)

Topic: Introduction to DBMS



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Content

- Difference between Schema and Instance in DBMS
- Data Independence
- Database Models
- DBMS Vs RDBMS

Difference between Schema and Instance in DBMS

Difference between Schema and Instance in DBMS

Instances:

- Instances are the collection of information stored at a particular moment.
- The instances can be changed by certain CRUD (Create, Read, Update, Delete) operations as like addition, deletion of data.

Example –

Let's say a table TEACHER in our database whose name is SCHOOL, suppose the table has 50 records so the instance of the database has 50 records for now and tomorrow we are going to add another fifty records so tomorrow the instance have total 100 records. This is called an instance.

Difference between Schema and Instance in DBMS

Schema :

- Schema is the overall description of the database.
- The basic structure of how the data will be stored in the database is called schema.

Example –

Let's say a table named TEACHER in our database name SCHOOL, the TEACHER table require the name, dob, doj in the table so we design a structure as :

TEACHER

name: varchar doj: date dob: date

Above given is the schema of the table TEACHER.

Data Independence

Data Independence

- Data independence can be explained using the three-schema architecture.

- Data independence refers characteristic of being able to modify the schema at one level of the database system without altering the schema at the next higher level.

- There are two types of data independence:

  (1) Logical Data Independence and

  (2) Physical Data Independence

Data Independence

Logical Data Independence:

- Logical data independence refers characteristic of being able to change the conceptual schema without having to change the external schema.
- Logical data independence is used to separate the external level from the conceptual view.
- If we do any changes in the conceptual view of the data, then the user view of the data would not be affected.
- Logical data independence occurs at the user interface level.

Data Independence

Physical Data Independence:

- Physical data independence can be defined as the capacity to change the internal schema without having to change the conceptual schema.
- If we do any changes in the storage size of the database system server, then the conceptual structure of the database will not be affected.
- Physical data independence is used to separate conceptual levels from the internal levels.
- Physical data independence occurs at the logical interface level.

Database Models

Database Models

- Data Model gives us an idea that how the final system will look like after its complete implementation.
- It defines the data elements and the relationships between the data elements.
- Data Models are used to show how data is stored, connected, accessed and updated in the DBMS.
- Some of the Data Models in DBMS are:

Hierarchical Model,
Entity-Relationship Model,
Flat Data Model,
Context Data Model,
Object-Relational Data Model,
Network Model,
Relational Model,
Object-Oriented Data Model,
Associative Data Model,
Semi-Structured Data Model etc.

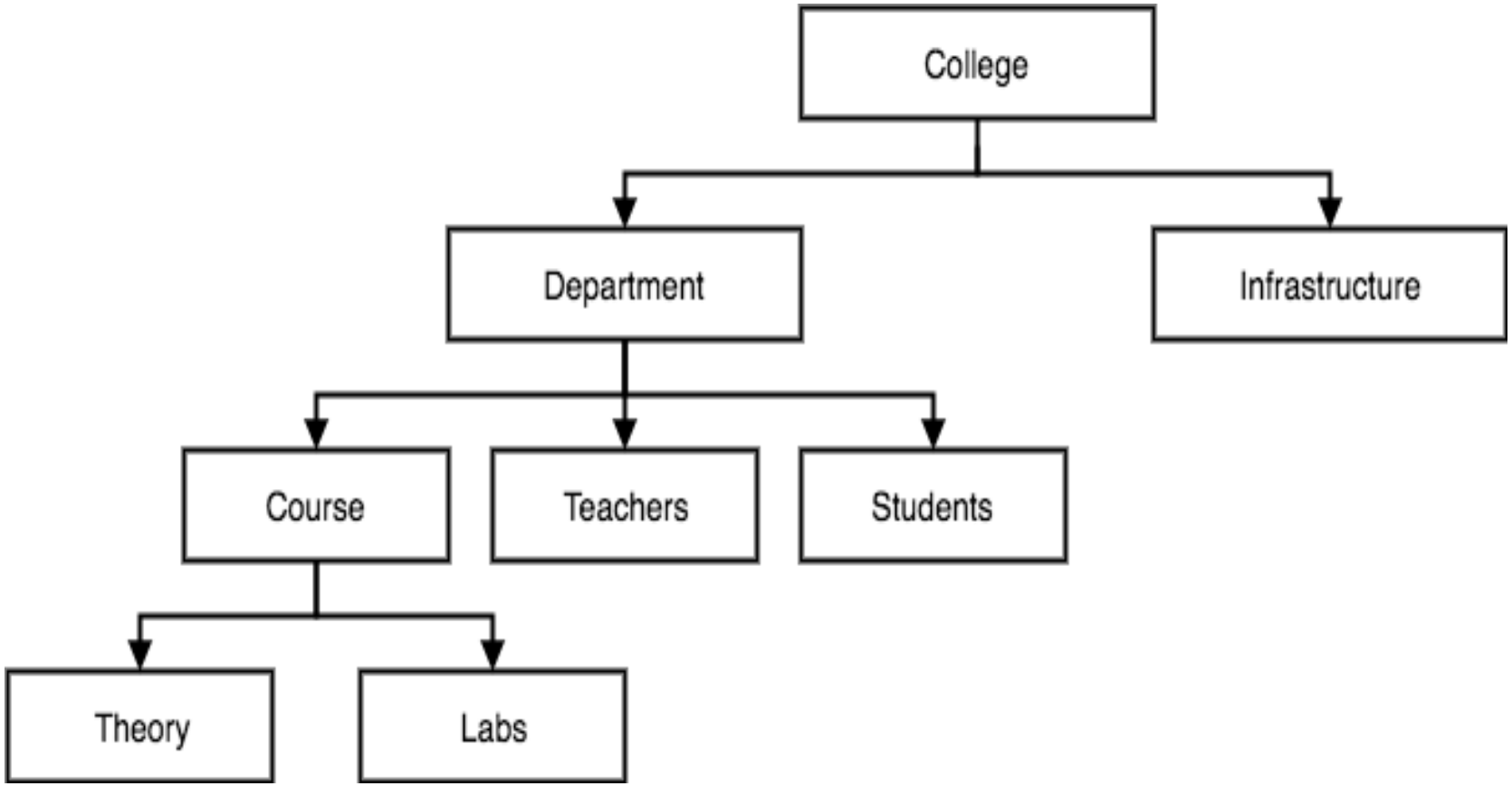
Database Models

Hierarchical Model:

- Hierarchical Model was the first DBMS model.
- This model organizes the data in the hierarchical tree structure.
- The hierarchy starts from the root which has root data and then it expands in the form of a tree adding child node to the parent node.
- This model easily represents some of the real-world relationships like hierarchy of any organization, sitemap of a website etc.

Database Models

Example:



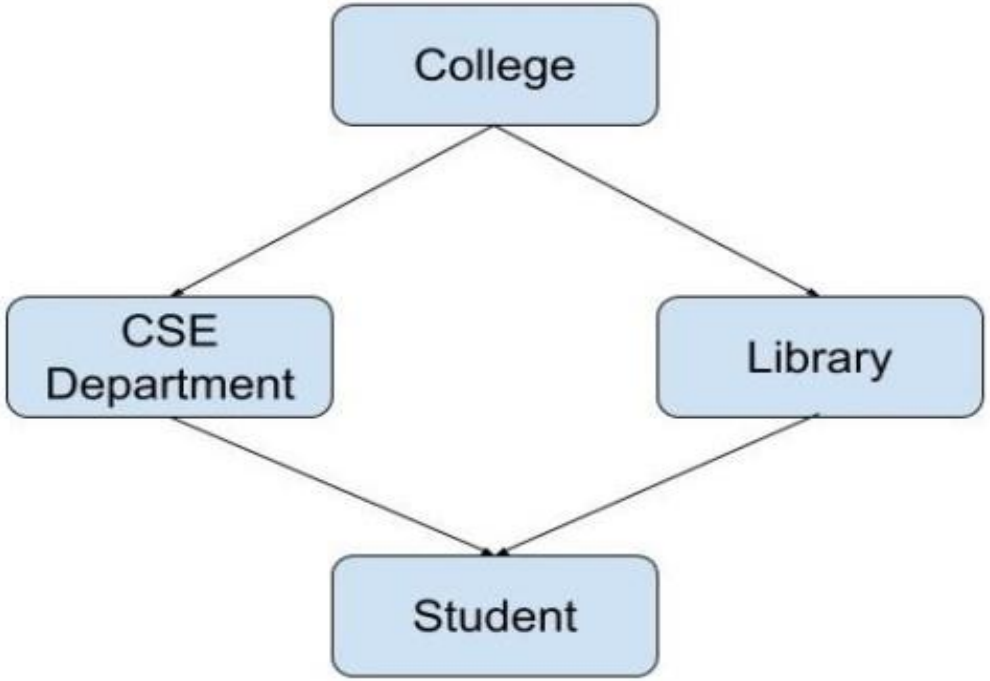
Database Models

Network Model

- This model is an extension of the hierarchical model.
- It was the most popular model before the relational model.
- This model is the same as the hierarchical model, the only difference is that a record can have more than one parent.
- It replaces the hierarchical tree with a graph.

Database Models

Example: In the example below we can see that node student has two parents i.e. CSE Department and Library. This was earlier not possible in the hierarchical model.



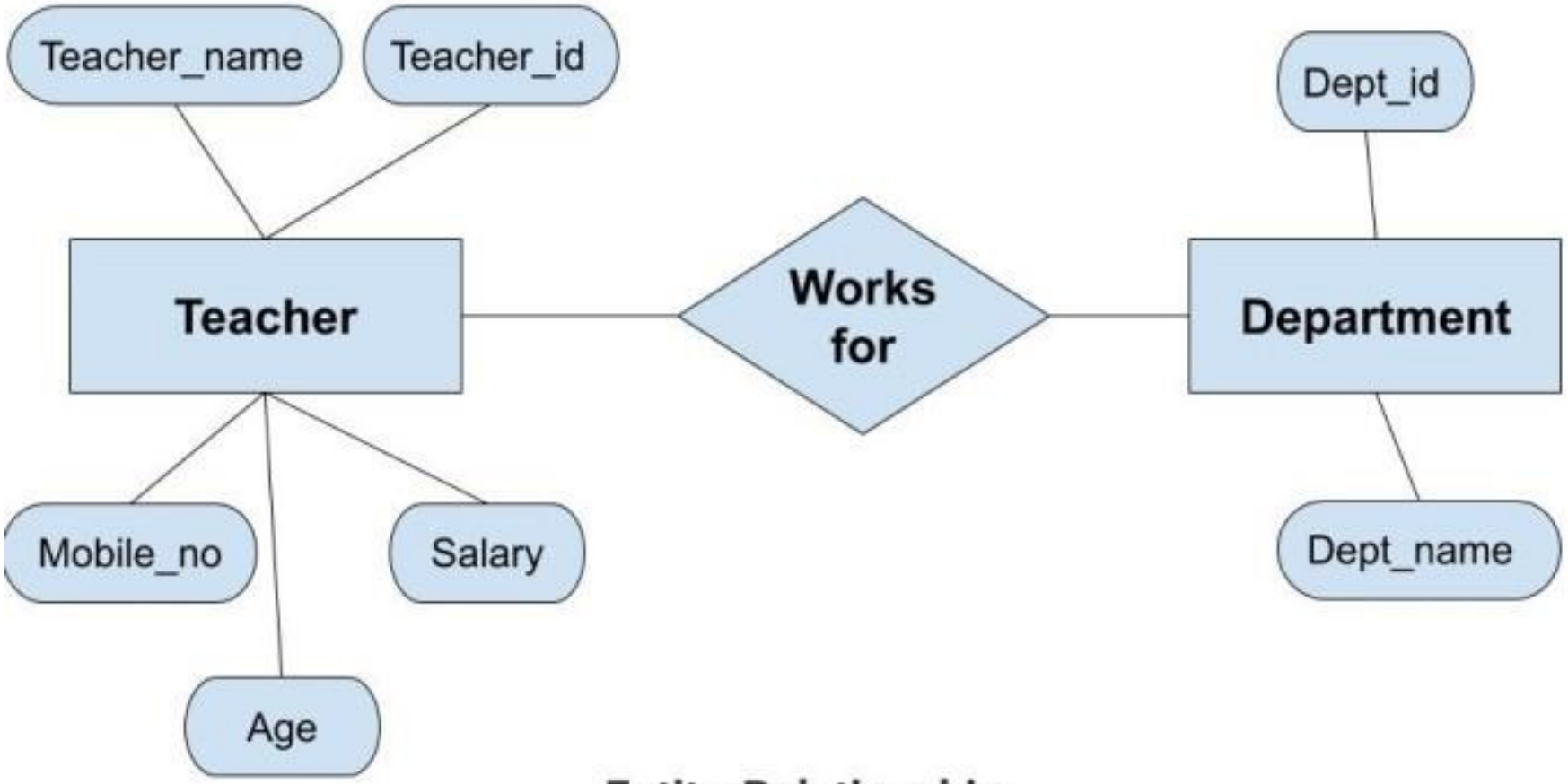
Network Model

Database Models

Entity-Relationship Model

- Entity-Relationship Model or simply ER Model is a high-level data model diagram.
- In this model, we represent the real-world problem in the pictorial form to make it easy for the stakeholders to understand.
- It is also very easy for the developers to understand the system by just looking at the ER diagram.
- We use the ER diagram as a visual tool to represent an ER Model.
- ER diagram has the following three components: Entity, Attribute and Relationship.

Database Models



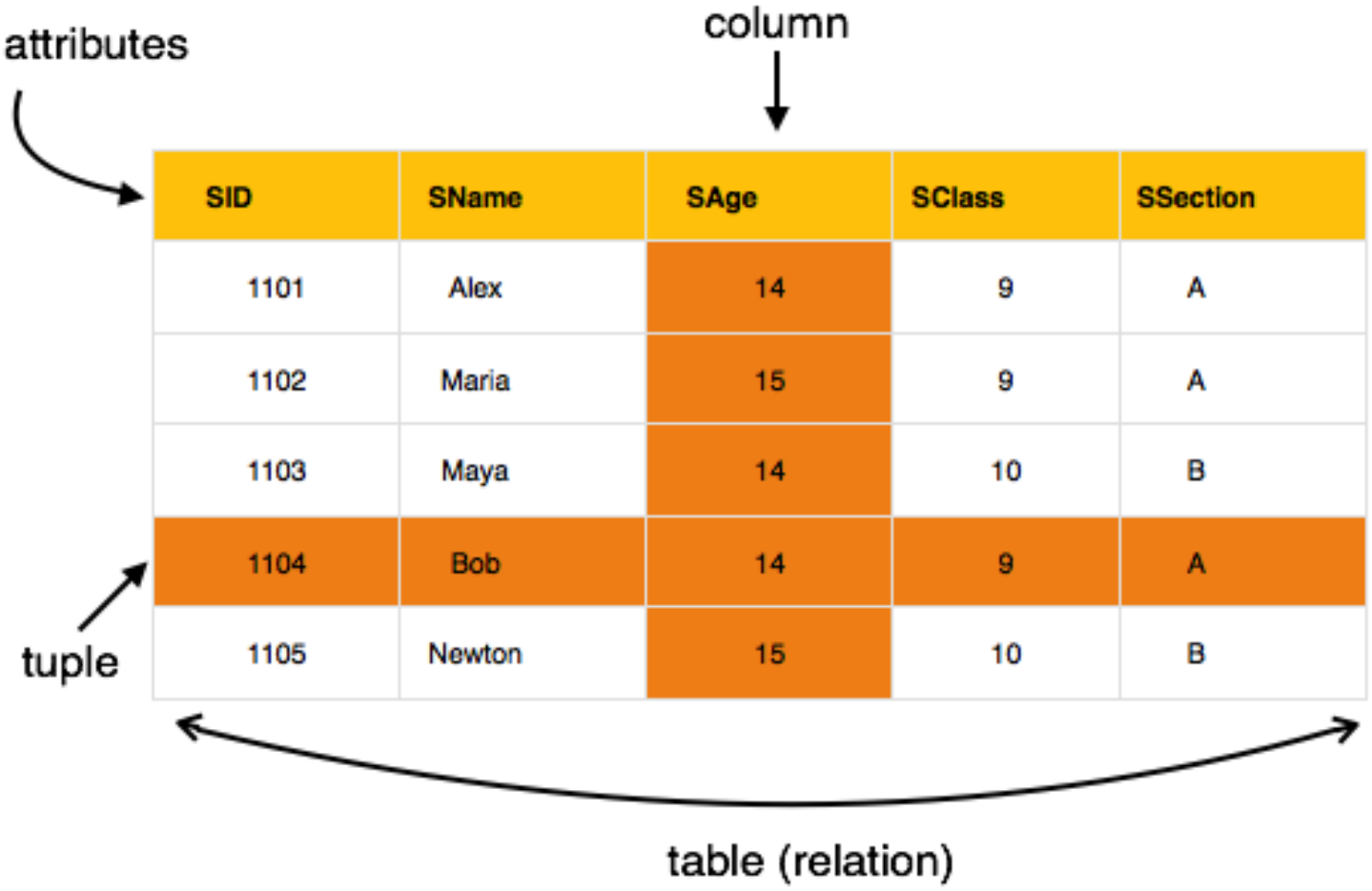
Entity-Relationship Model

Database Models

Relational Model

- This model was introduced by Dr. E. F. Codd in 1970.
- Relational Model is the most widely used model.
- In this model, the data is maintained in the form of a two-dimensional table.
- All the information is stored in the form of rows and columns.
- The basic structure of a relational model is tables.
- So, the tables are also called relations in the relational model.

Database Models



DBMS Vs RDBMS

DBMS Vs RDBMS

The below table demonstrates the main difference between RDBMS and DBMS:

DBMS	RDBMS
DBMS stores data as a file.	Data is stored in the form of tables.
Data elements need to access individually.	Multiple data elements can be accessed at the same time.
No relationship between data.	Data is stored in the form of tables, which are related to each other.
It deals with small quantity of data.	It deals with large amount of data.
Data redundancy is common in this model.	Keys and indexes do not allow Data redundancy.
It supports single user.	It supports multiple users.
The data in a DBMS is subject to low security levels with regards to data manipulation.	There exists multiple levels of data security in a RDBMS.
Examples: XML, Window Registry, etc.	Examples: MySQL, PostgreSQL, SQL Server, Oracle, Microsoft Access etc.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 5th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 6th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.geeksforgeeks.org/difference-between-schema-and-instance-in-dbms/last accessed on 10 December 2021.
- https://www.javatpoint.com/dbms-data-independence/last accessed on 10 December 2021.


Thank
you

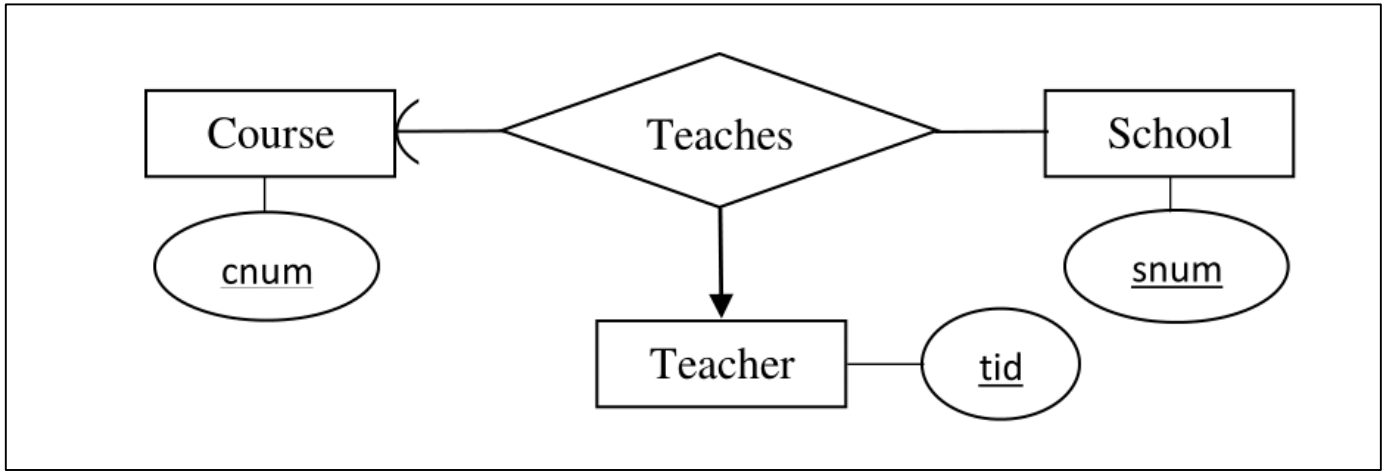
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management Systems (BCSC-0003)

Topic: E–R Diagrams



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

E–R Diagram

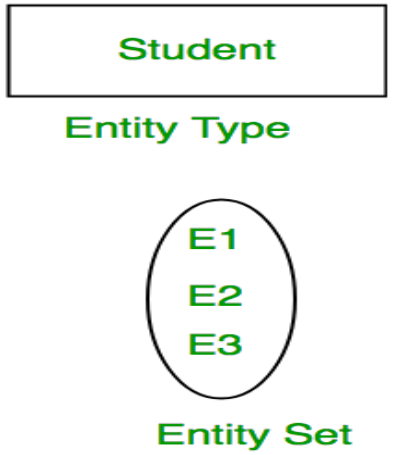
- E–R diagram stands for Entity Relationship Diagram.
- Entity–relationship modeling was developed for database and design by Peter Chen in 1976.
- An E–R diagram shows the relationship among entity sets.
- An entity set is a group of similar entities and these entities can have attributes.
- In terms of DBMS, an entity is a table.
- By showing relationship among tables and their attributes, E–R diagram shows the complete logical structure of a database.

Introduction of E–R Model

ER Model is used to model the logical view of the system from data perspective which consists of these components:

Entity, Entity Type, Entity Set:

- An Entity may be an object with a physical existence – a particular person, car, house, or employee – or it may be an object with a conceptual existence – a company, a job, or a university course.
- An Entity is an object of Entity Type and set of all entities is called as entity set. e.g.; E1 is an entity having Entity Type Student and set of all students is called Entity Set. In ER diagram, Entity Type is represented as:



Introduction of E–R Model

Attribute:

- Attributes are the properties which define the entity type. For example, Roll_No, Name, DOB, Age, Address, Mobile_No are the attributes which defines entity type Student. In ER diagram, attribute is represented by an oval.



Introduction of E–R Model

1. Key Attribute –

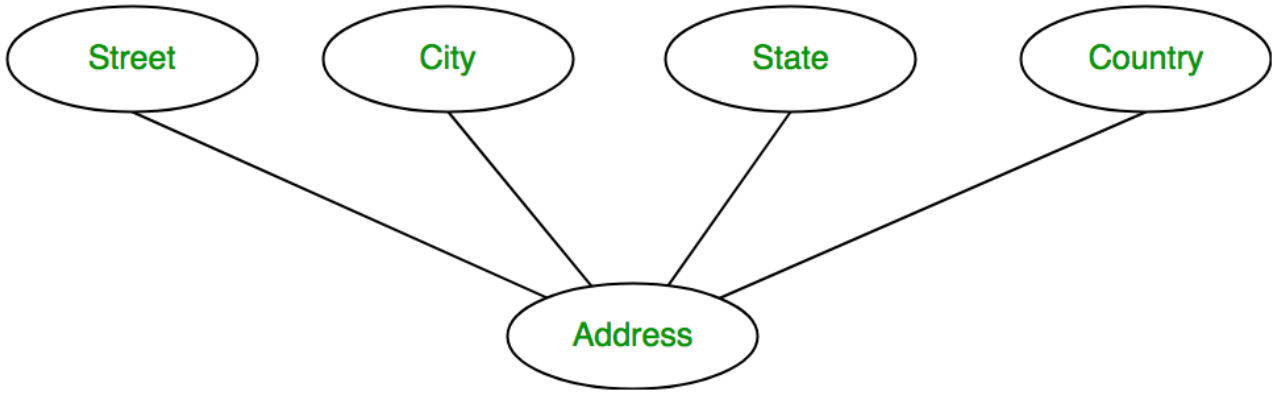
The attribute which uniquely identifies each entity in the entity set is called key attribute. For example, Roll_No will be unique for each student. In ER diagram, key attribute is represented by an oval with underlying lines.



Introduction of E–R Model

2. Composite Attribute –

An attribute composed of many other attribute is called as composite attribute. For example, Address attribute of student Entity type consists of Street, City, State, and Country. In ER diagram, composite attribute is represented by an oval comprising of ovals.



Introduction of E–R Model

3. Multivalued Attribute –

An attribute consisting more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, multivalued attribute is represented by double oval.



Introduction of E–R Model

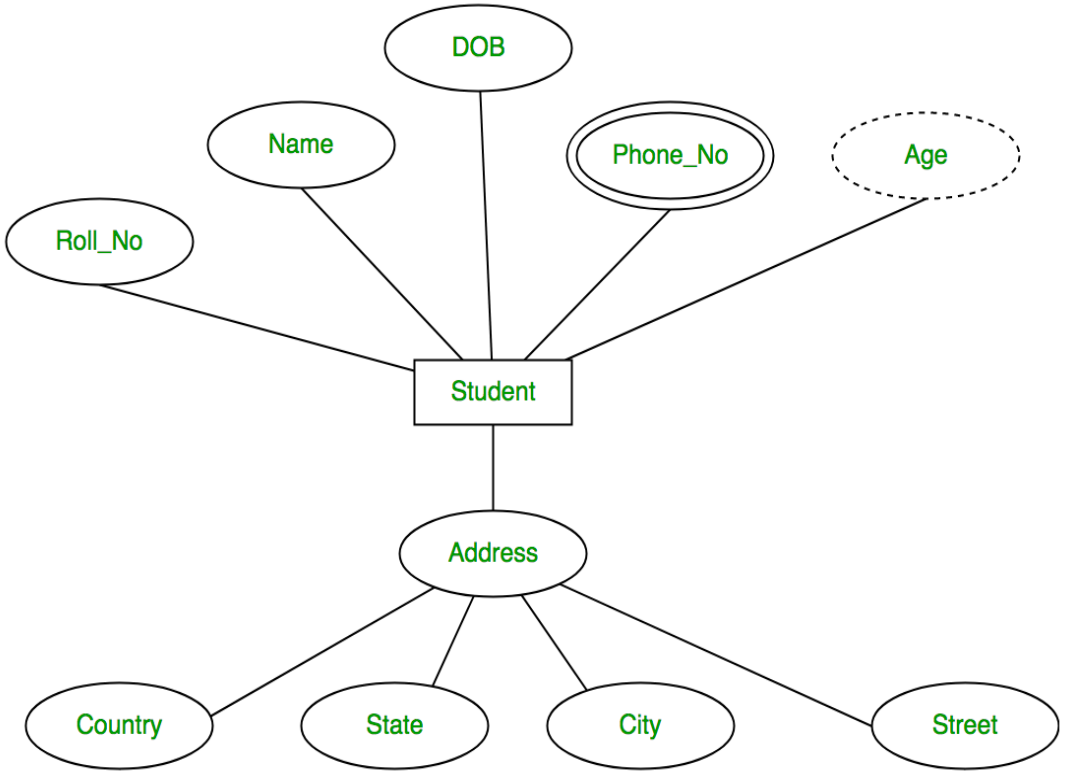
4. Derived Attribute –

An attribute which can be derived from other attributes of the entity type is known as derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, derived attribute is represented by dashed oval.



Introduction of E–R Model

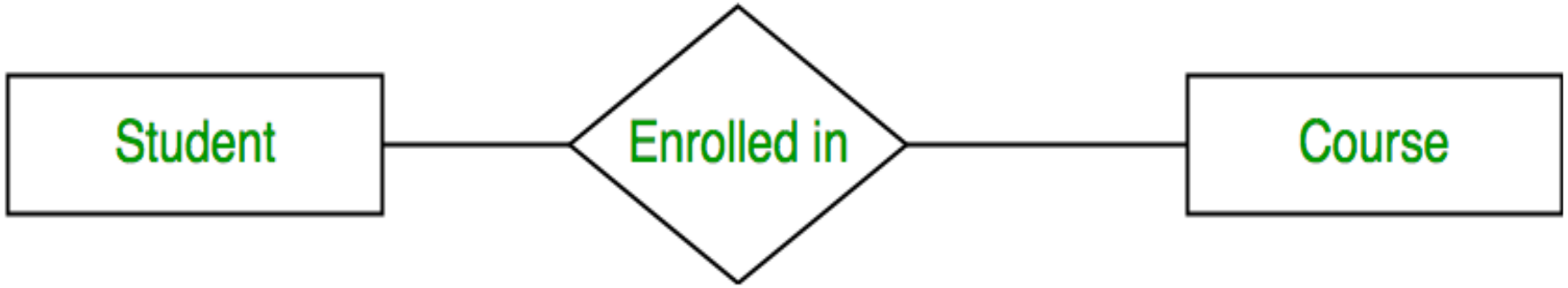
The complete entity type Student with its attributes can be represented as:



Introduction of E–R Model

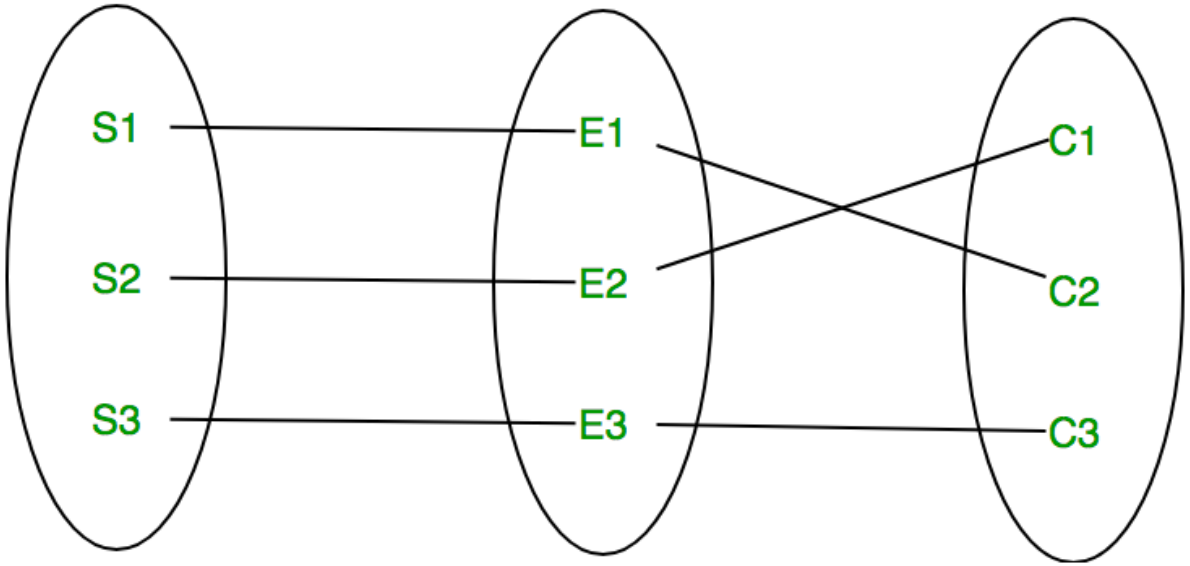
Relationship Type and Relationship Set:

A relationship type represents the association between entity types. For example, ‘Enrolled in’ is a relationship type that exists between entity type Student and Course. In ER diagram, relationship type is represented by a diamond and connecting the entities with lines.



Introduction of E–R Model

A set of relationships of same type is known as relationship set. The following relationship set depicts S1 is enrolled in C2, S2 is enrolled in C1 and S3 is enrolled in C3.



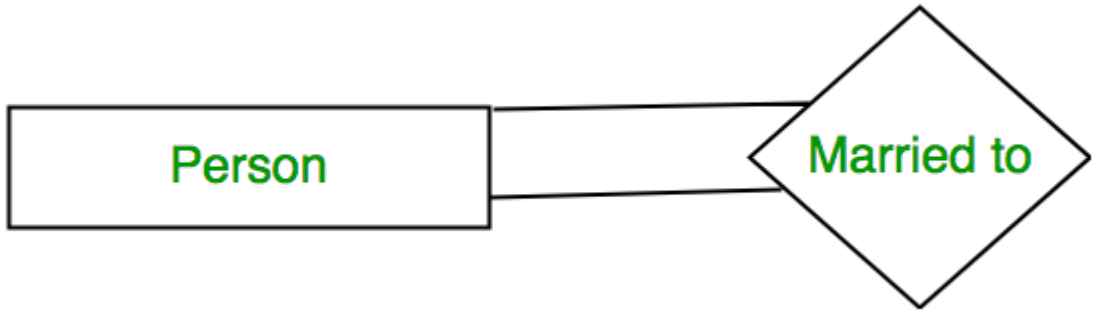
Introduction of E–R Model

Degree of a relationship set:

The number of different entity sets participating in a relationship set is called as degree of a relationship set.

1. Unary Relationship –

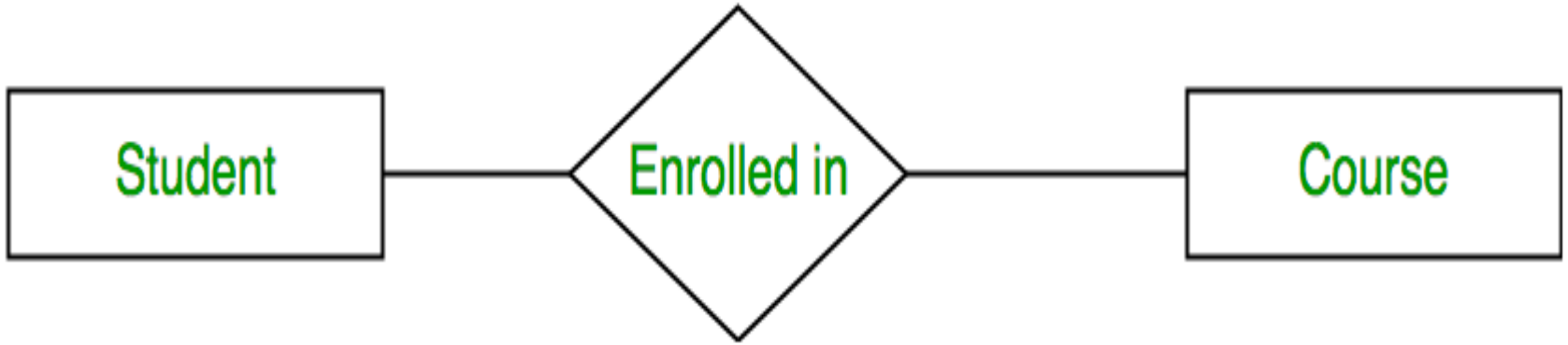
When there is only ONE entity set participating in a relation, the relationship is called as unary relationship. For example, one person is married to only one person.



Introduction of E–R Model

2. Binary Relationship –

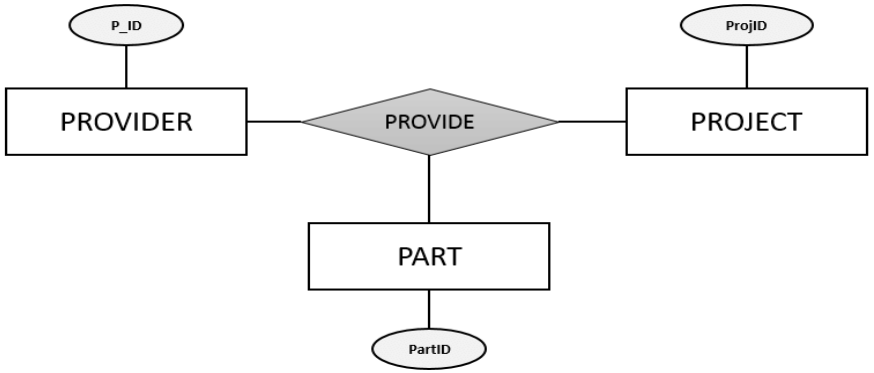
When there are TWO entities set participating in a relation, the relationship is called as binary relationship. For example, Student is enrolled in Course.



Introduction of E–R Model

3. n-ary Relationship –

When there are n entities set participating in a relation, the relationship is called as n-ary relationship.



Cardinality:

The number of times an entity of an entity set participates in a relationship set is known as cardinality. Cardinality can be of different types as:

A. One to One Relationship

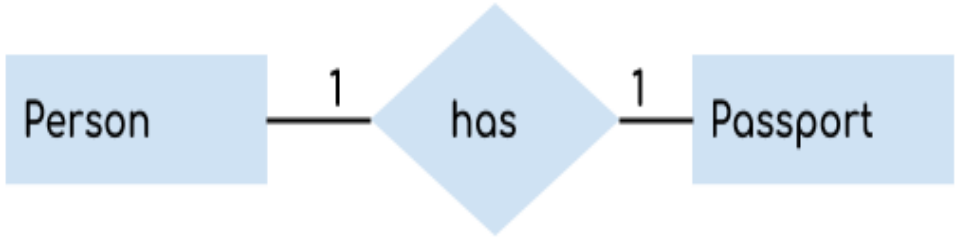
B. One to Many Relationship

C. Many to One Relationship &

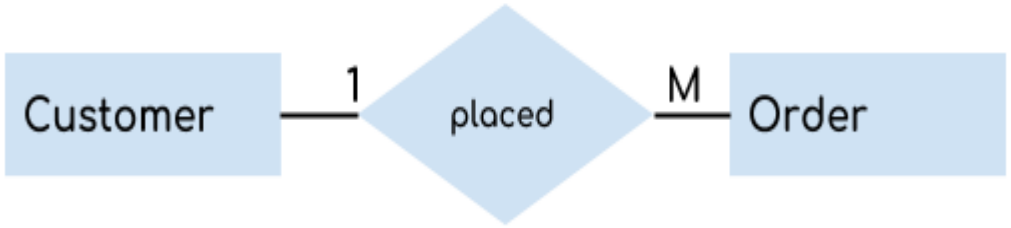
D. Many to Many Relationship

Introduction of E–R Model

A. One to One Relationship – When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport and a passport is given to one person.



B. One to Many Relationship – When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship. For example – a customer can place many orders but a order cannot be placed by many customers.



Introduction of E–R Model

C. Many to One Relationship – When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship. For example – many students can study in a single college but a student cannot study in many colleges at the same time.



D. Many to Many Relationship – When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship. For example, a student can be assigned to many projects and a project can be assigned to many students.



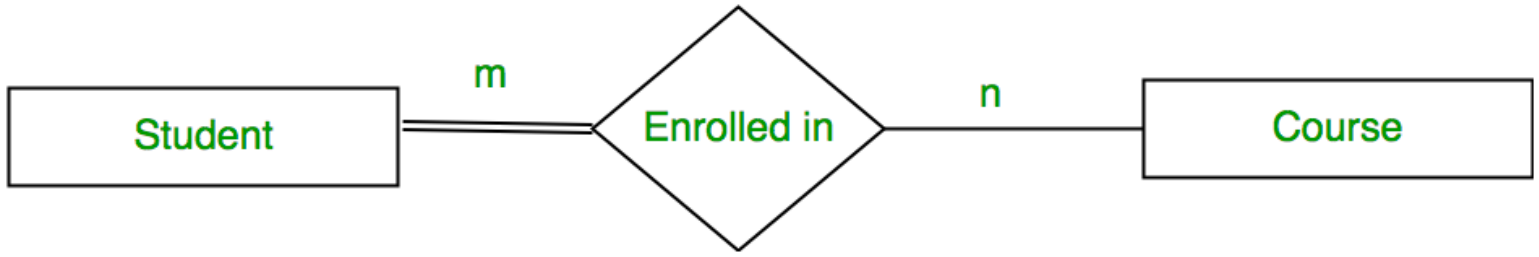
Introduction of E–R Model

Participation Constraint:

Participation Constraint is applied on the entity participating in the relationship set.

Total Participation – Each entity in the entity set must participate in the relationship. If each student must enroll in a course, the participation of student will be total. Total participation is shown by double line in ER diagram.

Partial Participation – The entity in the entity set may or may NOT participate in the relationship. If some courses are not enrolled by any of the student, the participation of course will be partial.



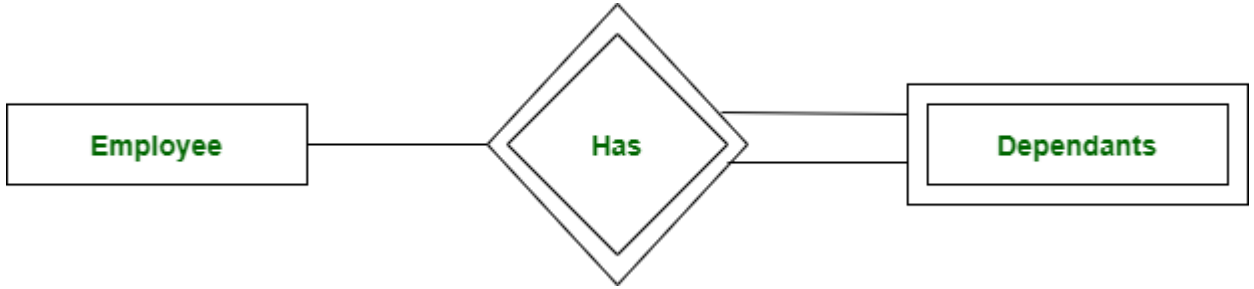
The diagram depicts the ‘Enrolled in’ relationship set with Student Entity set having total participation and Course Entity set having partial participation.

Introduction of E–R Model

Weak Entity Type and Identifying Relationship:

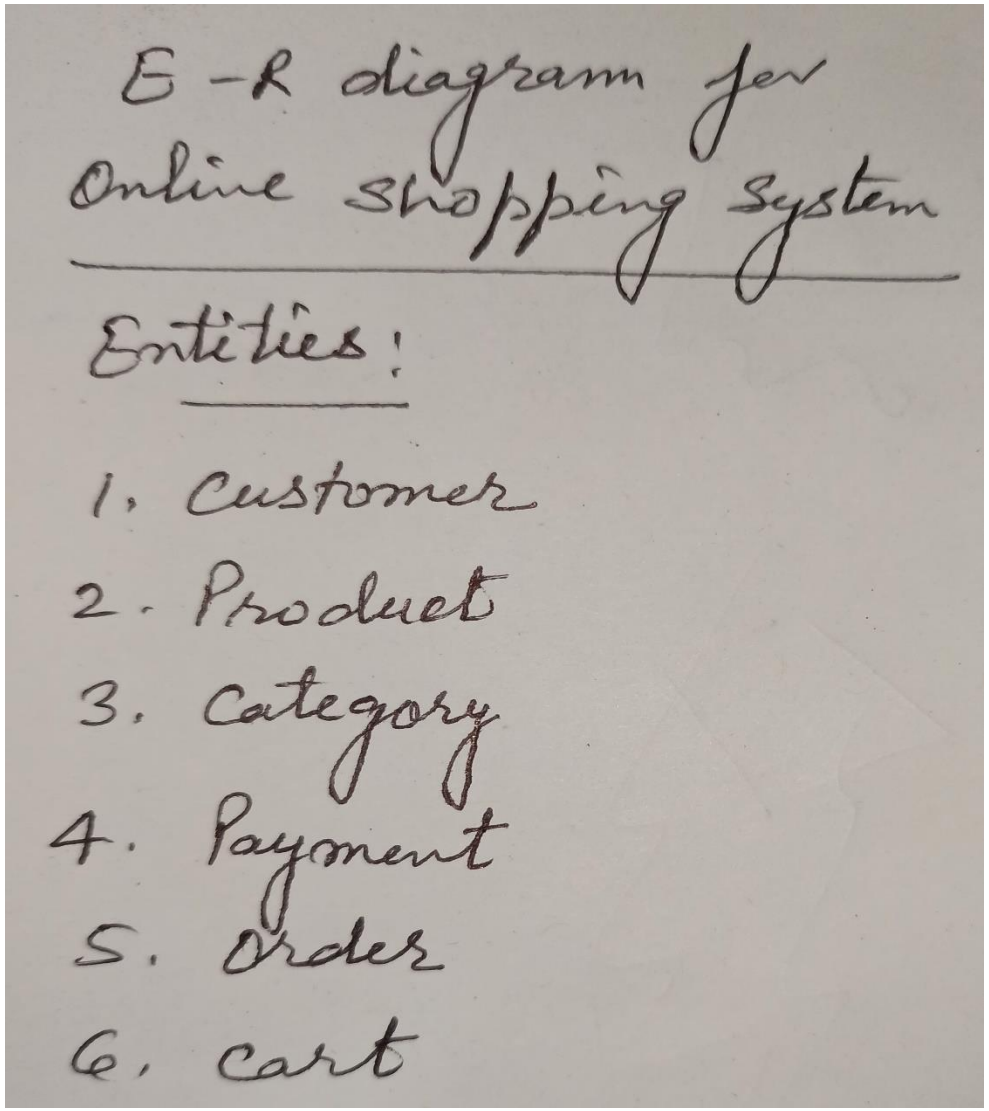
As discussed before, an entity type has a key attribute which uniquely identifies each entity in the entity set. But there exists some entity type for which key attribute can’t be defined. These are called Weak Entity type.

For example, A company may store the information of dependants (Parents, Children, Spouse) of an Employee. But the dependents don’t have existence without the employee. So Dependent will be weak entity type and Employee will be Identifying Entity type for Dependant.

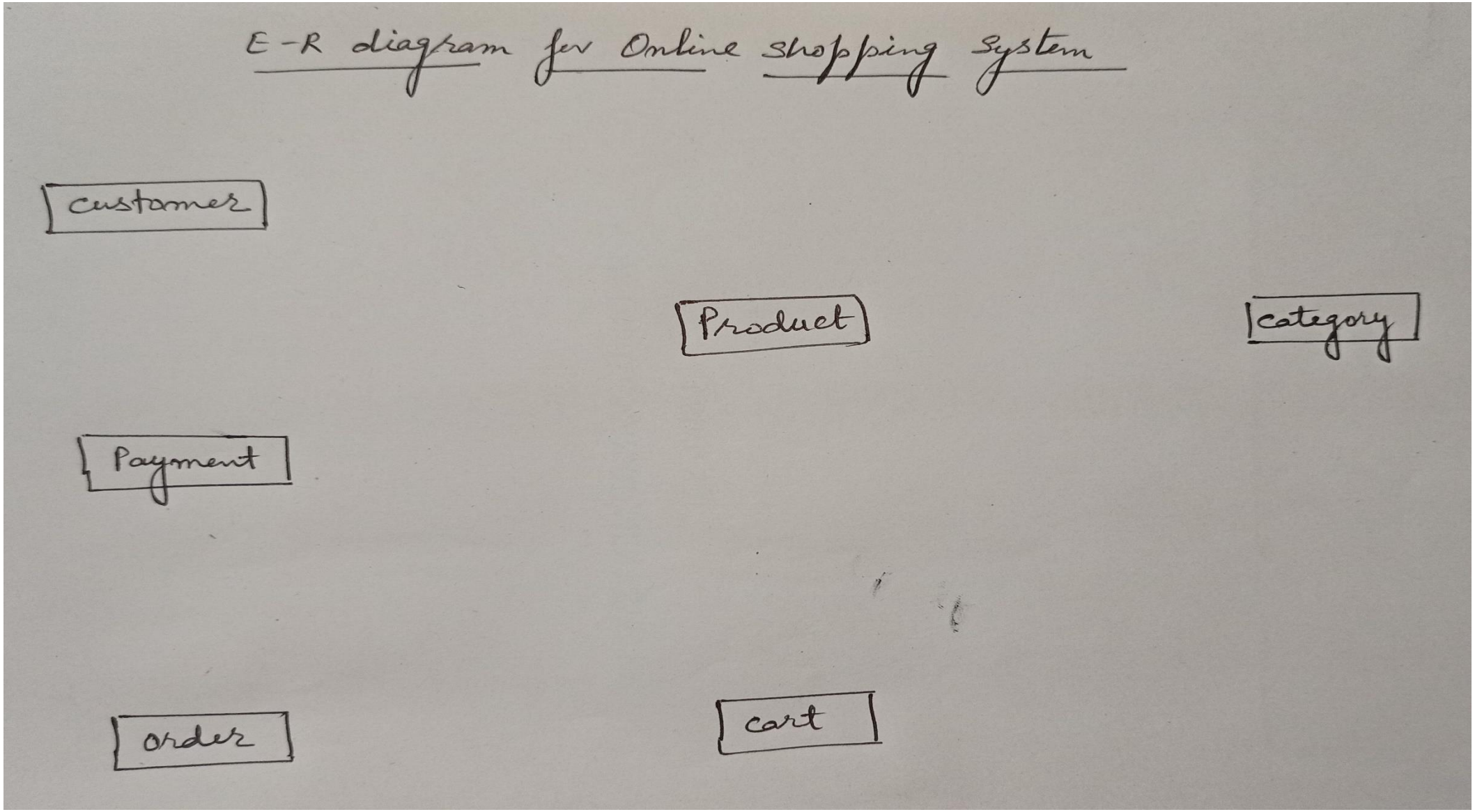


A weak entity type is represented by a double rectangle. The participation of weak entity type is always total. The relationship between weak entity type and its identifying strong entity type is called identifying relationship and it is represented by double diamond.

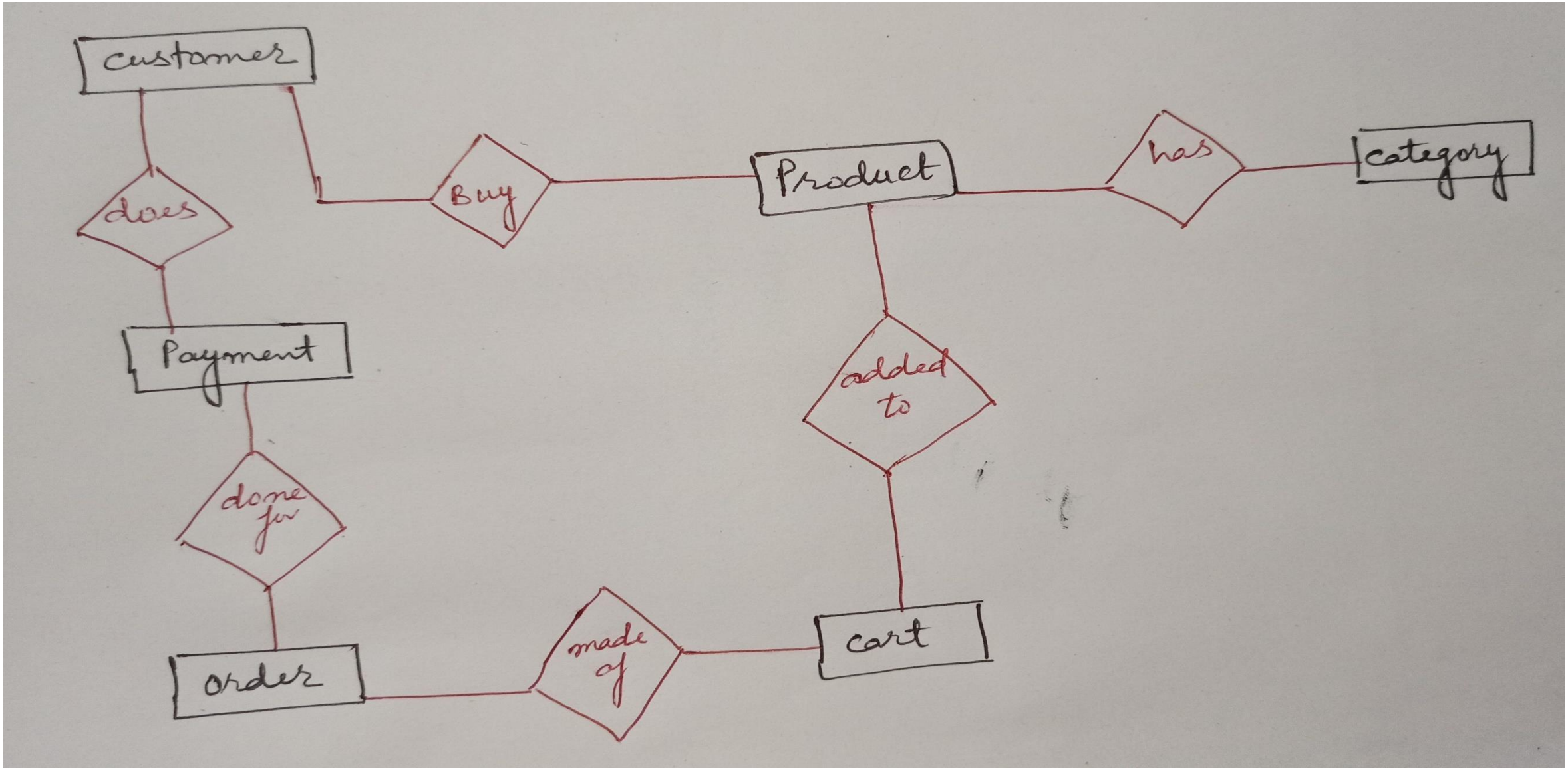
E–R Diagram for Online Shopping System



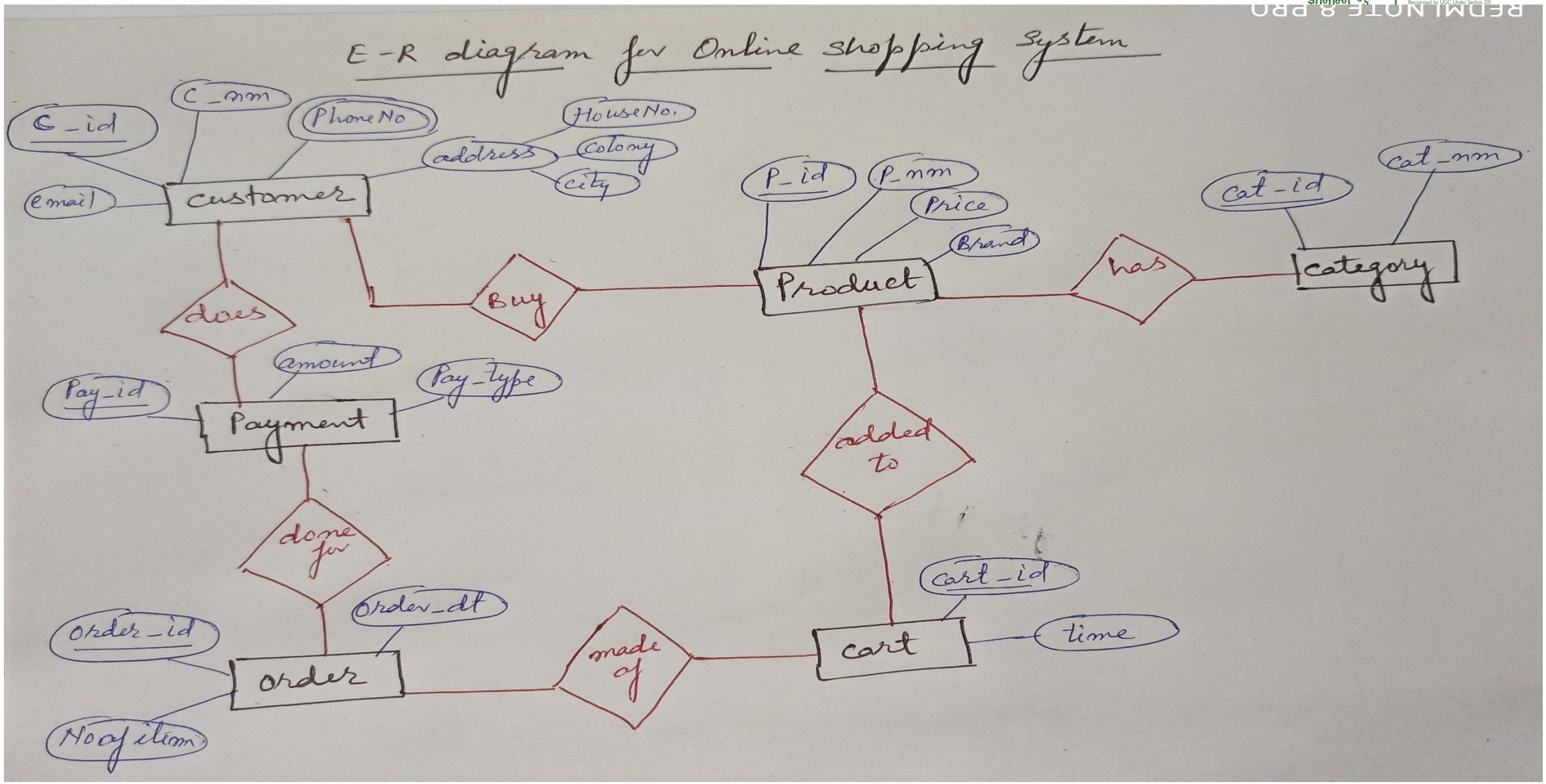
E–R Diagram for Online Shopping System



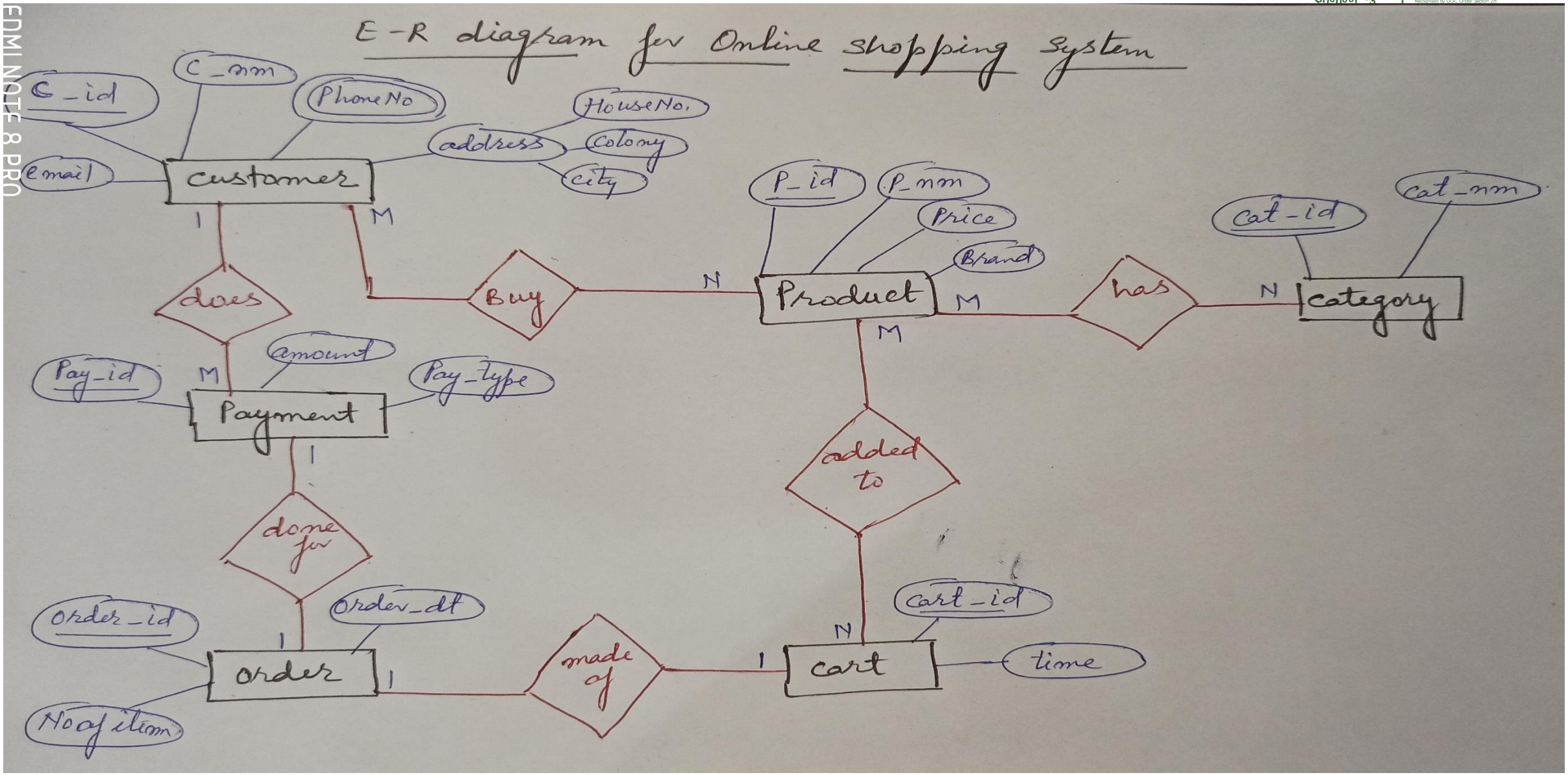
E–R Diagram for Online Shopping System



E–R Diagram for Online Shopping System



E–R Diagram for Online Shopping System



References

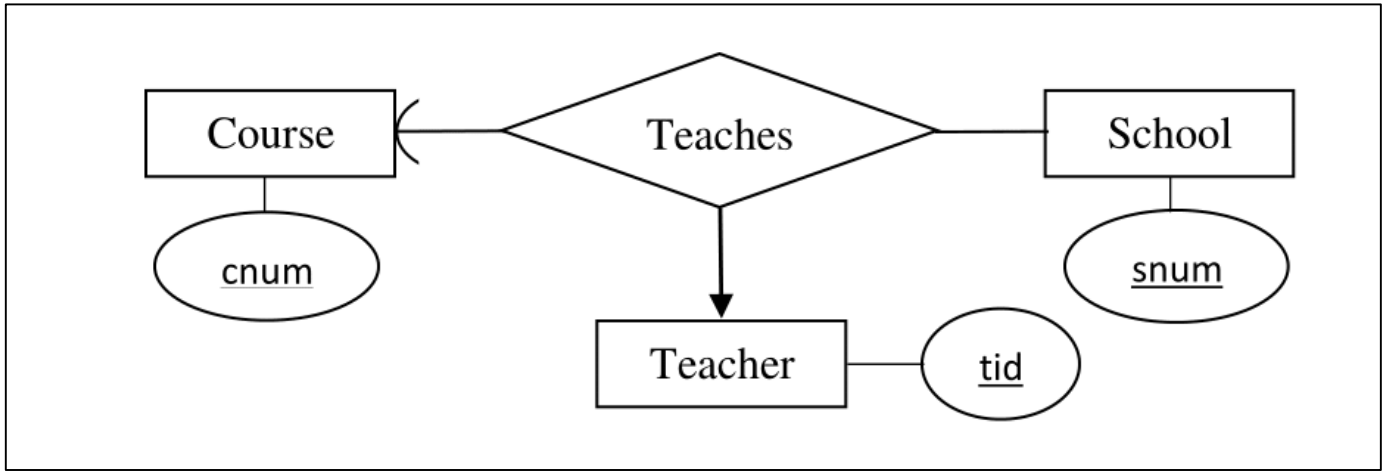
- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you

GLA UNIVERSITY MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A Grade by NAAC
12-B Status from UGC

Database Management Systems (BCSC-1003)

Topic: EER Diagrams



Dr. Nikhil Govil

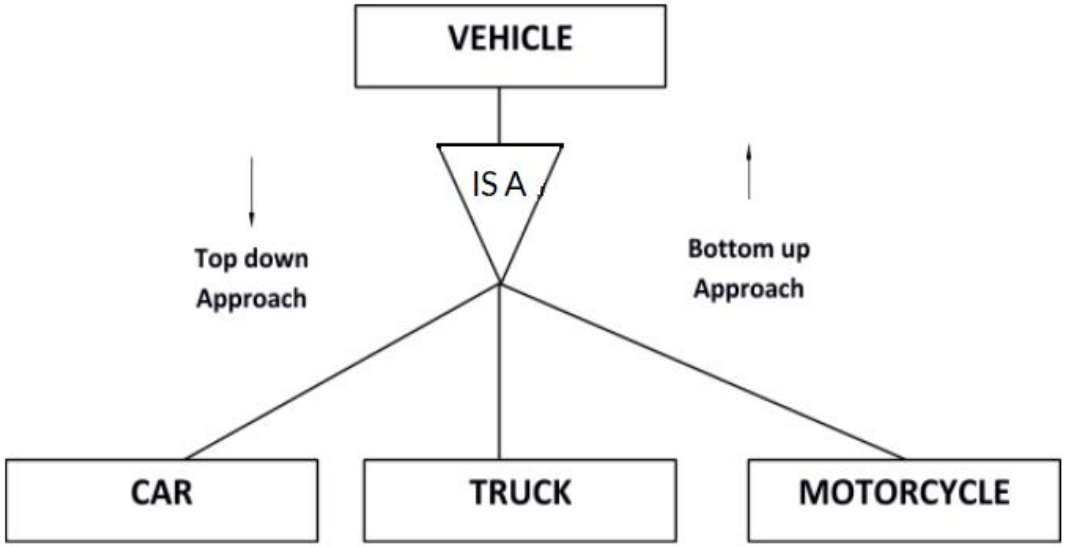
Assistant Professor, Dept. of CEA, GLA University, Mathura.

EER Diagram

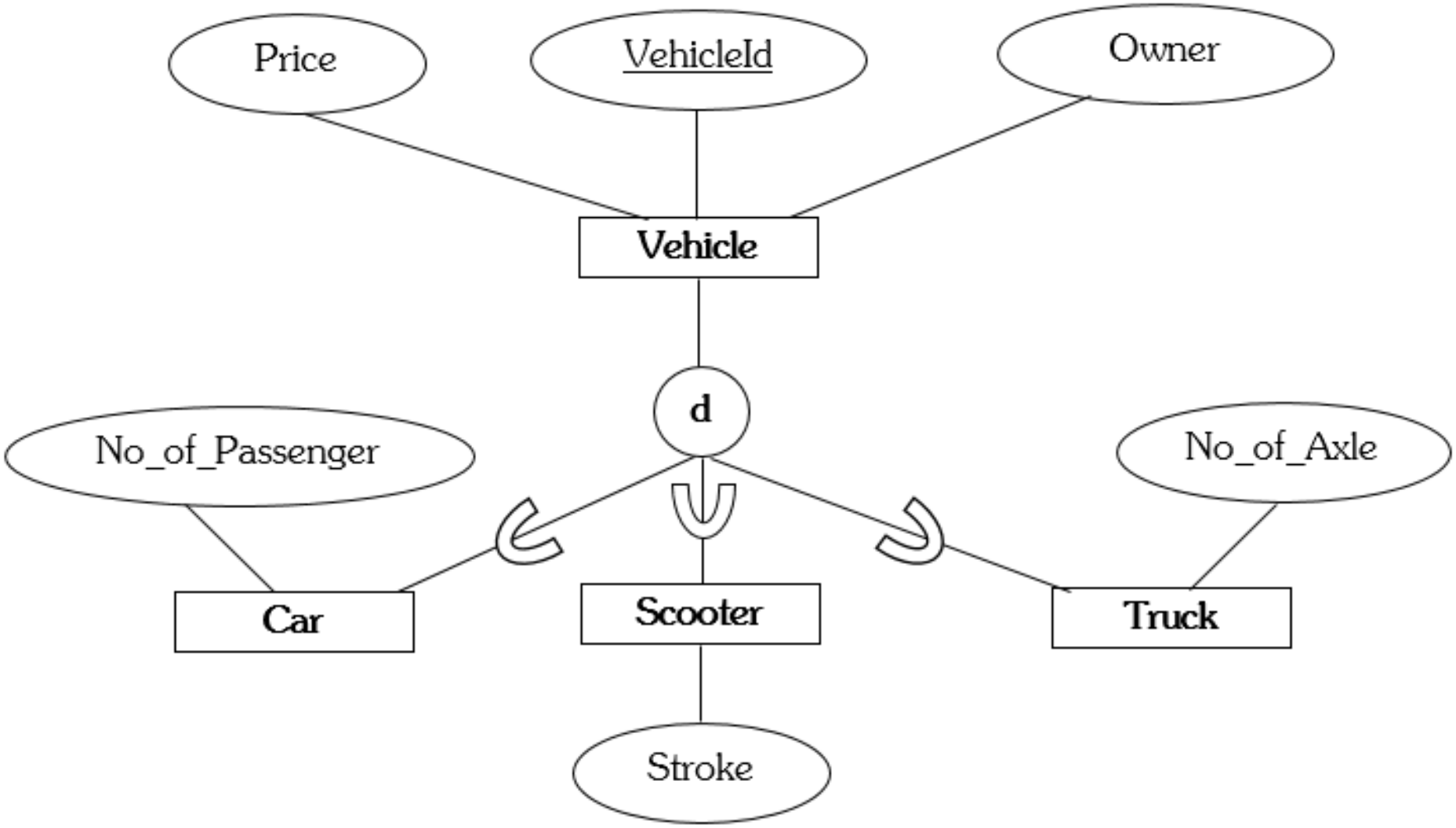
- EER diagram stands for Enhanced (or Extended) Entity Relationship Diagram.
- EER is a high-level data model that incorporates the extensions to the original ER model.
- Enhanced or Extended ER Diagrams are high level models that represent the requirements and complexities of complex database.
- In addition to ER model concepts EER includes Specialization, Generalization and Aggregation.

Generalization & Specialization

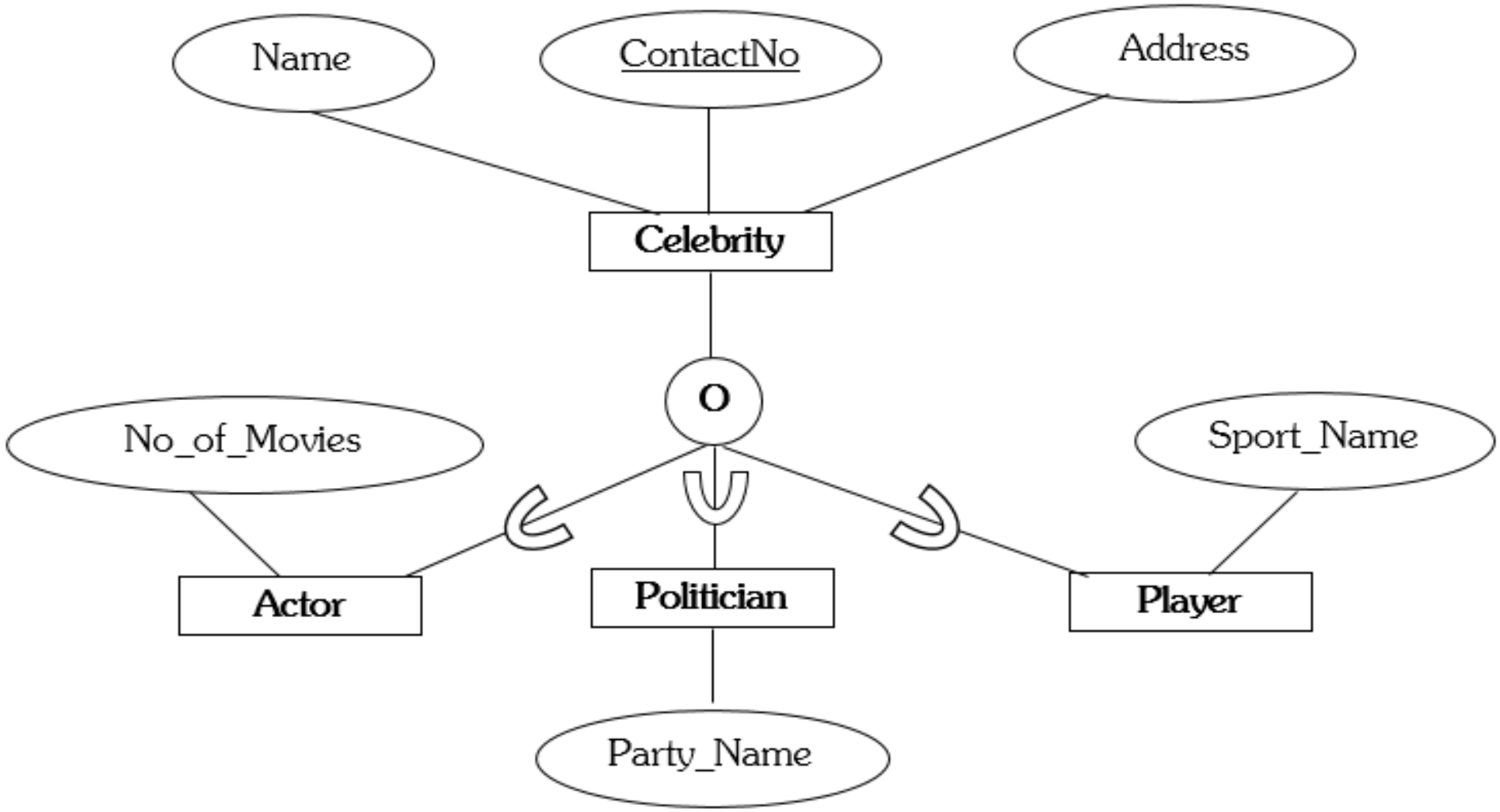
- **Generalization:** Generalization is a process of generalizing an entity which contains generalized attributes or properties of generalized entities. It is a Bottom up process i.e. consider we have 3 sub entities Car, Truck and Motorcycle. Now these three entities can be generalized into one super class named as Vehicle.
- **Specialization:** Specialization is a process of identifying subsets of an entity that share some different characteristic. It is a top down approach in which one entity is broken down into low level entity.



Generalization & Specialization



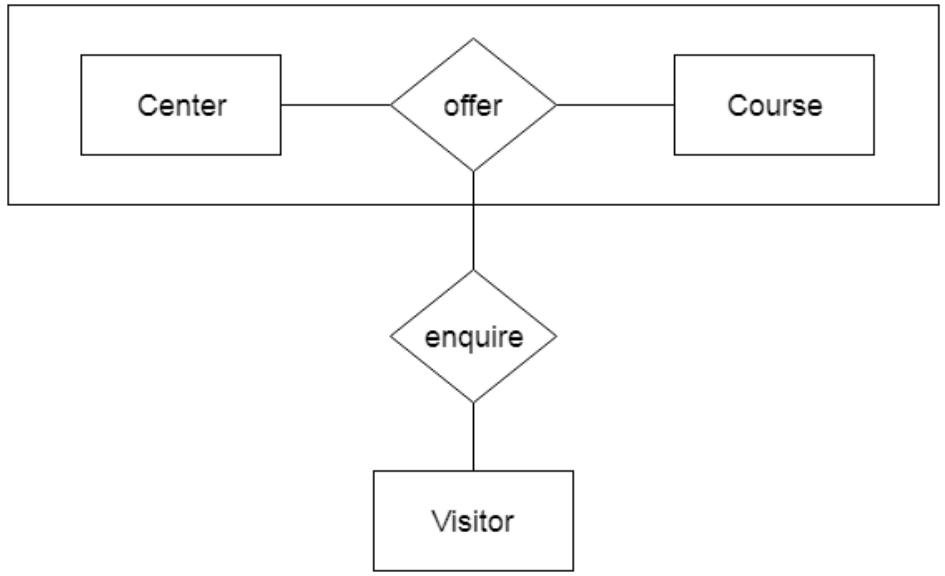
Generalization & Specialization



Aggregation

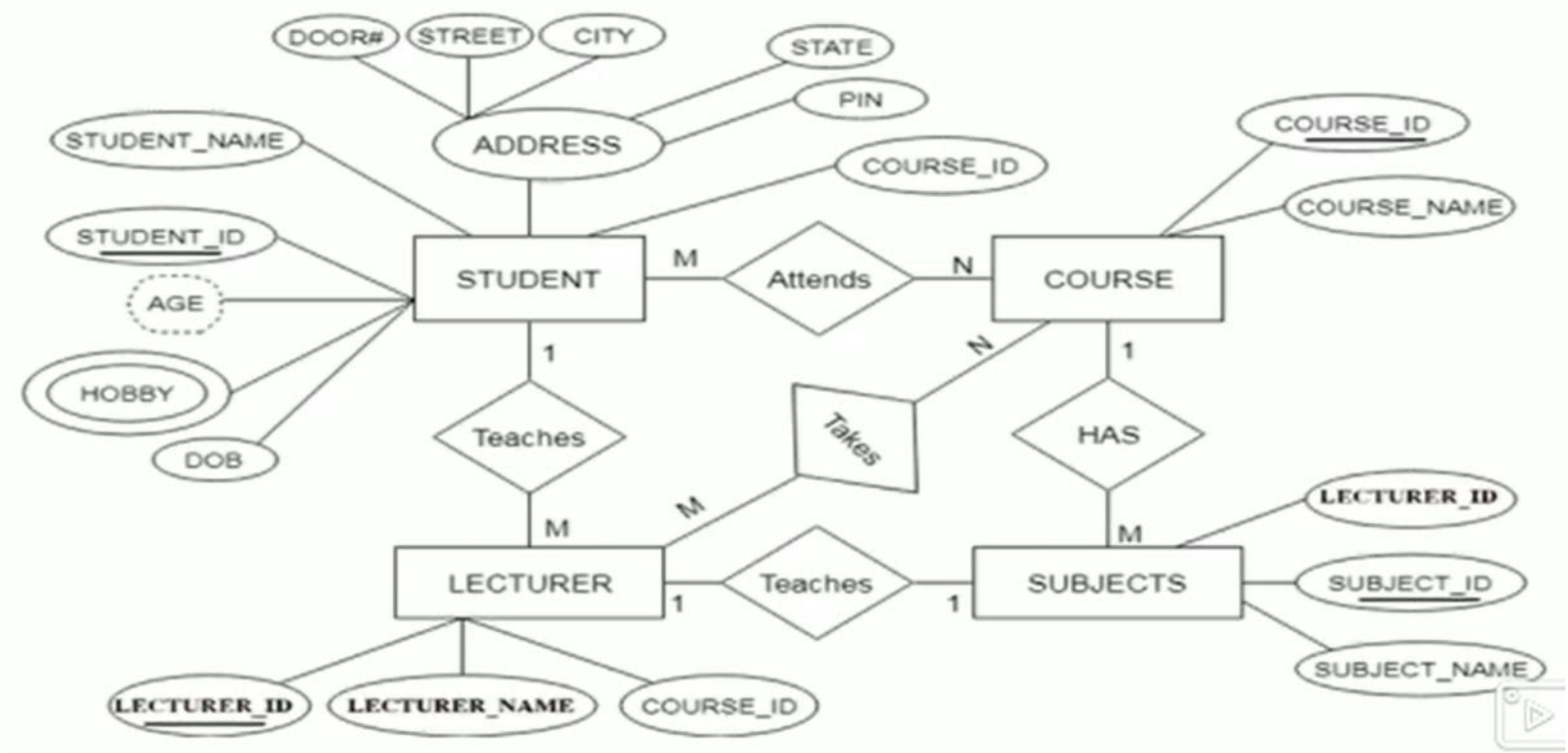
- In aggregation, the relation between two entities is treated as a single entity. In aggregation, relationship with its corresponding entities is aggregated into a higher level entity.

Example:

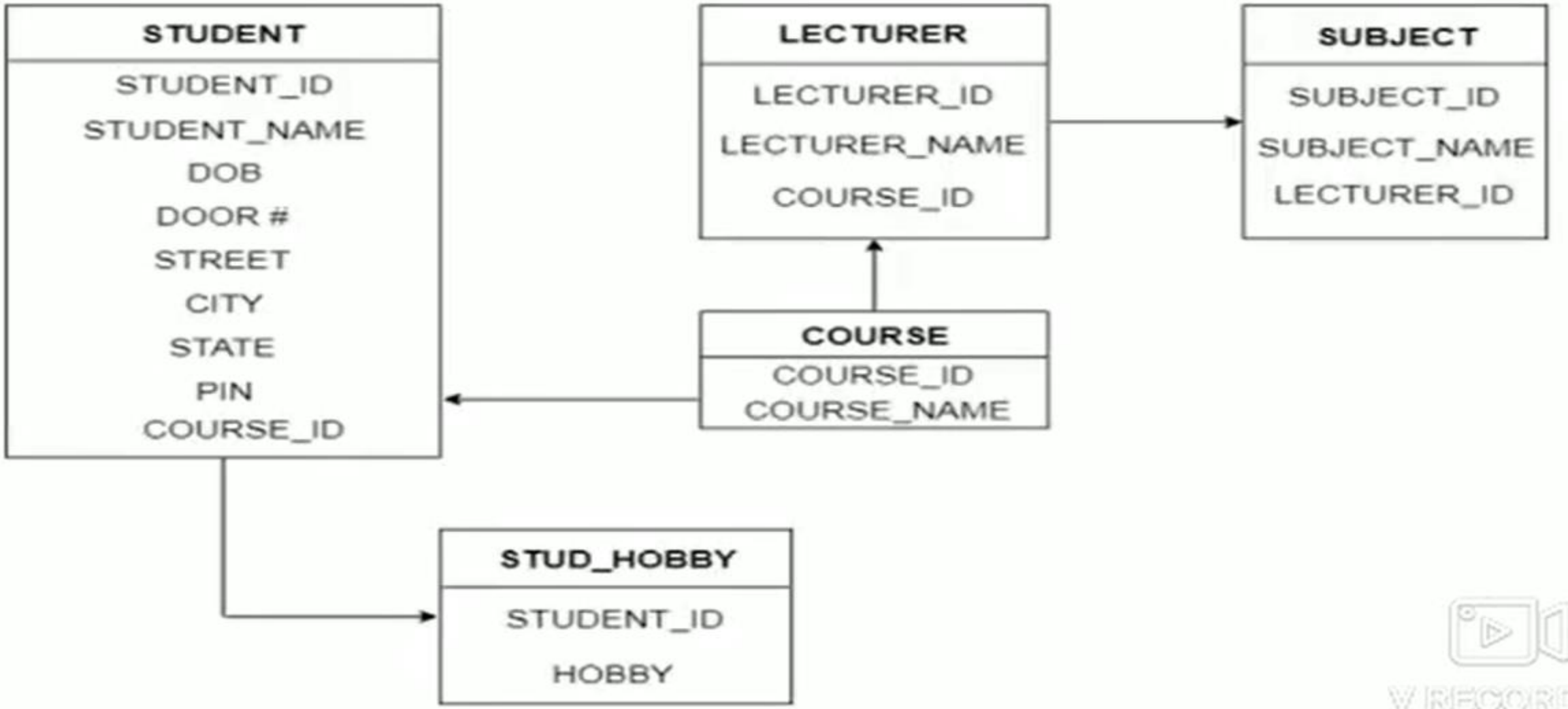


Center entity offers the Course entity act as a single entity in the relationship which is in a relationship with another entity visitor. In the real world, if a visitor visits a coaching center then he will never enquiry about the Course only or just about the Center instead he will ask the enquiry about both.

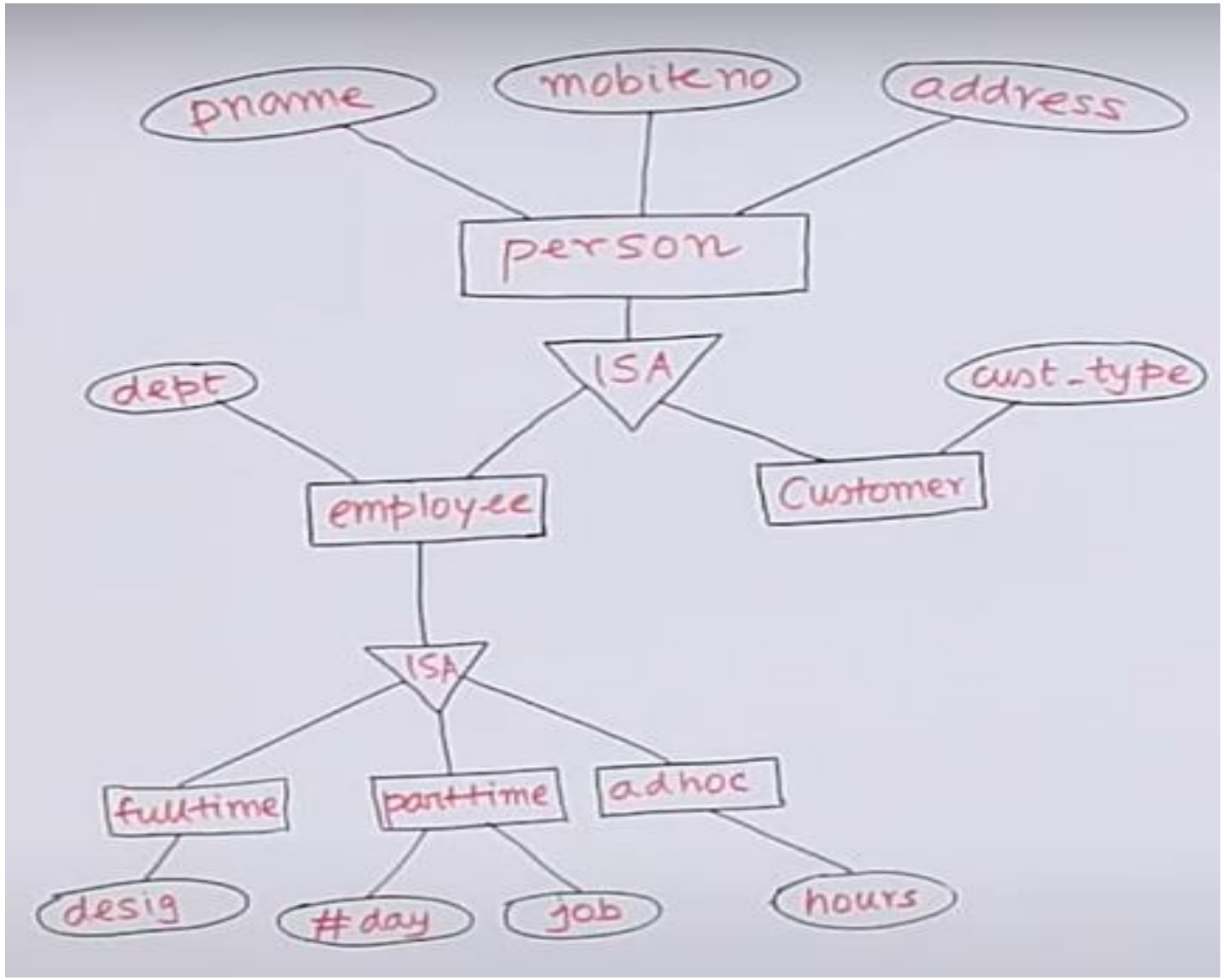
Converting E-R Diagrams into Table



Converting E-R Diagrams into Table



Converting EER Diagrams into Table



Converting EER Diagrams into Table

Tables will be:

1. CUSTOMER (pname, mobileno, address, cust_type)

2. FULLTIME (pname, mobileno, address, dept, desig)

3. PARTTIME (pname, mobileno, address, dept, #days, job)

4. ADHOC (pname, mobileno, address, dept, hours)

References

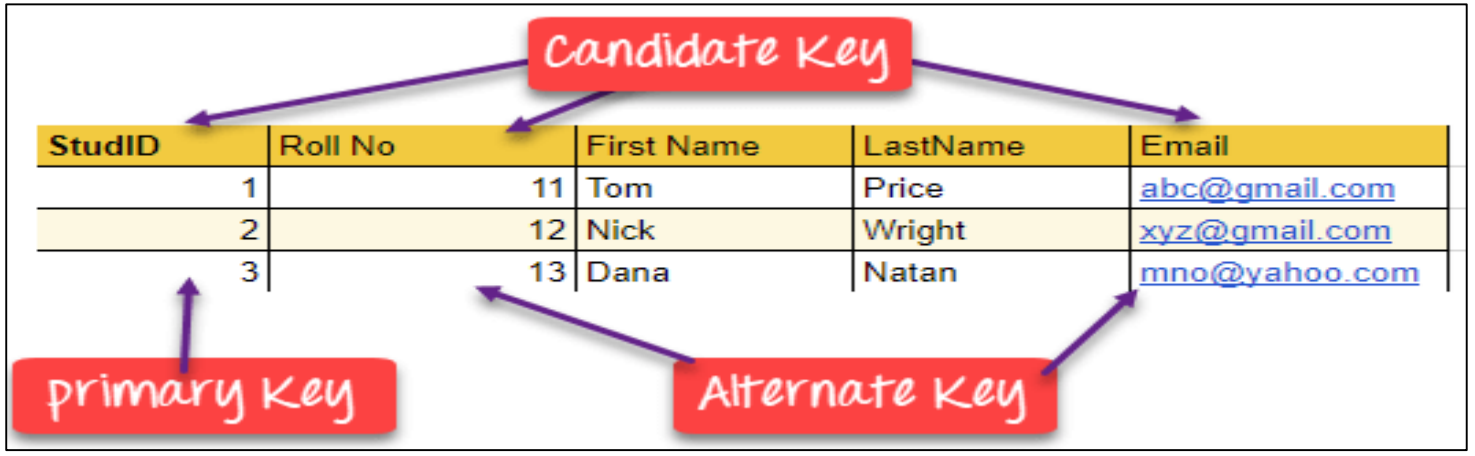
- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-aggregation/last accessed on 12 December’ 2022.


Thank
you

GLA UNIVERSITY MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A Grade by NAAC
12-B Status from UGC

Database Management Systems (BCSC-0003)

Topic: DBMS Keys



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

What are the Keys in DBMS?

- A key in DBMS is an attribute or a set of attributes that help to uniquely identify a tuple (or row) in a relation (or table).

- Keys are also used to establish relationships between the different tables and columns of a relational database. Individual values in a key are called key values.

Why are the Keys Required?

- A key is used in the definitions of various kinds of integrity constraints.
- A table in a database represents a collection of records or events for a particular relation.
- Now there can be thousands and thousands of such records, some of which may be duplicated.
- There should be a way to identify each record separately and uniquely, i.e. no duplicates.
- Keys allow us to be free from this hassle.
- A key could either be a combination of more than one attribute (or columns) or just a single attribute.
- The main motive of this is to give each record a unique identity.

Types of Keys in DBMS

There are broadly nine types of keys in DBMS:

1. Primary Key
2. Candidate Key or Secondary Key
3. Alternate Key
4. Super Key
5. Composite Key
6. Unique Key
7. Surrogate Key or Artificial Key
8. Foreign Key
9. Partial Key

1. Primary Key (PK)

- A Primary Key is a column of a table or a set of columns that helps to identify every record present in that table uniquely.
- There can be only one Primary Key in a table.
- Also, the Primary Key cannot have the same values repeating for any row.
- Every value of the Primary Key has to be different with no repetitions.
- The Primary Key constraint put on a column or set of columns will not allow them to have any null values or any duplicates.

2. Candidate Key (CK) or Secondary Key

- Candidate Keys are those attributes that uniquely identify rows of a table.
- The PK of a table is selected from one of the Candidate Keys.
- So, Candidate Keys have the same properties as the PK.
- There can be more than one Candidate Keys in a table.

3. Alternate Key

- A table can have multiple choices for a PK; however, it can choose only one.

- So, all the keys which did not become the Primary Key are called Alternate Keys.

4. Super Key (SK)

- Super Key is the set of all the keys which help to identify rows in a table uniquely.

- This means that all those columns of a table than capable of identifying the other columns of that table uniquely will all be considered super keys.

- Super Key is the superset of a Candidate Key.

- The Primary Key of a table is picked from the Super Key set to be made the table's identity attribute.

5. Composite Key

- Composite Key is a set of two or more attributes that help identify each tuple in a table uniquely.
- The attributes in the set may not be unique when considered separately.
- However, when taken all together, they will ensure uniqueness.

6. Unique Key

- Unique Key is a column or set of columns that uniquely identify each record in a table.

- All values will have to be unique in this key.

- A Unique Key differs from a PK because it can have only one null value, whereas a PK cannot have any null values.

7. Surrogate Key or Artificial Key

- Surrogate Keys is an Artificial Key which aims to uniquely identify each record is called a Surrogate Key.
- This kind of key in DBMS is unique because it is created when we don’t have any natural PK.
- They do not lend any meaning to the data in the table.
- Surrogate Key in DBMS is usually an integer.
- Surrogate keys in SQL are allowed when:
  - ❖ No property has the parameter of the Primary Key.
  - ❖ In the table when the Primary Key is too big or complicated.

8. Foreign Key (FK)

- Foreign Key is used to establish relationships between two tables.
- A Foreign Key will require each value in a column or set of columns to match the Primary Key of the referential table.
- Foreign keys help to maintain data and referential integrity.

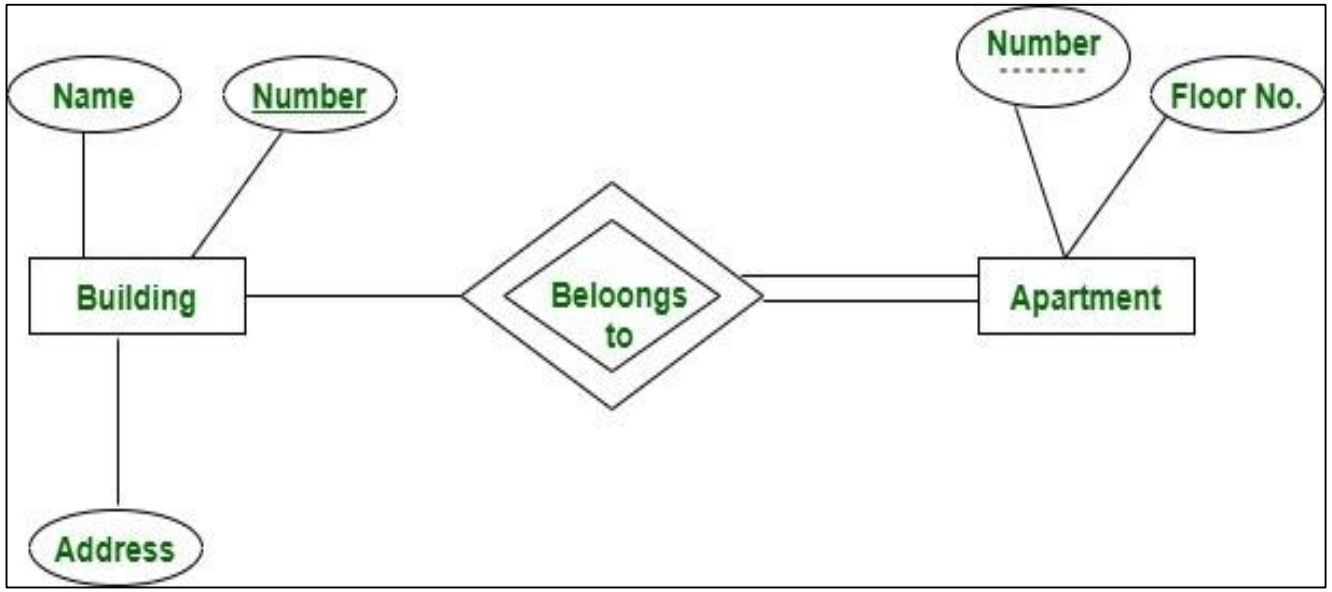
9. Partial Key

- The set of attributes that are used to uniquely identify a weak entity set is called the Partial key.

- Only a bunch of the tuples can be identified using the partial keys.

- The Partial Key of the weak entity set is also known as a discriminator.

9. Partial Key (Contd …)



Here we have an apartment as a weak entity and building as a strong entity type connected via ‘belongs to’ relationship set. Apartment number is not globally unique i.e. more than one apartment may have same number globally but it is unique for a particular building since a building may not have same apartment number. Thus apartment number cannot be primary key of entity Apartment but it is a partial key shown with a dashed line.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you



Database Management Systems (BCSC-1003)

Topic: Relational Algebra in DBMS

Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Relational Algebra

- Relational algebra is a widely used procedural query language.
- It takes instances of relations as input and yields instances of relations as output.
- It uses various operations to perform this action.
- Relational algebra mainly provides theoretical foundation for relational databases and SQL.

Basic SQL Relational Algebra Operations

Relational Algebra may be divided in various groups:

1. Relational Algebra Operations From Set Theory
    a. UNION (∪)
    b. INTERSECTION (∩)
    c. DIFFERENCE (–)

Basic SQL Relational Algebra Operations

2. Unary Relational Operations
    a. SELECT (σ)
    b. PROJECT (π)
    c. RENAME (ρ)

Basic SQL Relational Algebra Operations

3. Binary Relational Operations

I. JOIN (⋈)

Types of JOIN:

Various forms of join operation are:

A. Cross Join or Cartesian Product

B. Inner Joins:

(i). Theta join (ii). EQUI join (iii). Natural join

C. Outer join:

(i). Left Outer Join (ii). Right Outer Join

(iii). Full Outer Join

II. DIVISION (÷)

Relational Algebra Operations From Set Theory

UNION (∪)

UNION is symbolized by ∪ symbol. It includes all tuples that are in tables A or in B. It also eliminates duplicate tuples. So, set A UNION set B would be expressed as:

$R = A \cup B$

Relational Algebra Operations From Set Theory

UNION (∪)

Exp.

Relation A

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT

Relation B

CNAME	CSTATUS
KARAN	POOR
RAJAT	GOOD

$R = A \cup B$

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT
KARAN	POOR

Relational Algebra Operations From Set Theory

INTERSECTION (∩)

INTERSECTION is symbolized by ∩ symbol. Defines a relation consisting of a set of all tuple that are in both A and B. However, A and B must be union-compatible.

R = A ∩ B

Relational Algebra Operations From Set Theory

INTERSECTION (∩)

Exp.

Relation A

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT

Relation B

CNAME	CSTATUS
KARAN	POOR
RAJAT	GOOD

$R = A \cap B$

CNAME	CSTATUS
RAJAT	GOOD

Relational Algebra Operations From Set Theory

DIFFERENCE (–)

DIFFERENCE is symbolized by – symbol. The result of A–B, is a relation which includes all tuples that are in A but not in B.

R = A – B

Relational Algebra Operations From Set Theory

DIFFERENCE (–)

Exp.

Relation A

CNAME	CSTATUS
RAJAT	GOOD
RAHUL	EXCELLENT

Relation B

CNAME	CSTATUS
KARAN	POOR
RAJAT	GOOD

R = A – B

CNAME	CSTATUS
RAHUL	EXCELLENT

Unary Relational Operations

a. SELECT (σ)

b. PROJECT (π)

c. RENAME (ρ)

SELECT Operator (σ)

- SELECT operation is used for selecting a subset of the tuples according to a given selection condition.
- Select Operator is denoted by sigma (σ).
- It is used as an expression to choose tuples which meet the selection condition.
- Select operator selects tuples that satisfy a given predicate.

SELECT Operator (σ)

Syntax:

$\sigma_{<Condition>}$ (Relation/Table name)

Properties:

1. It is like a WHERE clause in SQL.
2. Selection operator is commutative i.e.,

$\sigma_{<condition_1>}(\sigma_{<condition_2>}(R)) = \sigma_{<condition_2>}(\sigma_{<condition_1>}(R))$

SELECT Operator (σ)

Examples: STUDENT

ROLL_NO	NAME	AGE	COURSE
1	ABHISHEK	17	BTech
2	AMIT	16	BCA
3	AJEET	17	BTech
4	AKHIL	18	BTech
5	PRASHANT	17	BCA

Query 1: Find out the students of course 'BTech'.

Syntax: $\sigma_{\text{course = 'BTech'}}$ (student)

Query2: Find the student(s) whose age is greater than 17.

Syntax: $\sigma_{\text{age > 17}}$ (student)

PROJECT or PROJECTION Operator (π)

- PROJECT Operator is denoted by pi (π).
- Project operation selects certain attributes discarding other attributes.
- It is also known as vertical partitioning since it partitions the relation or table vertically.
- Duplicate rows are automatically eliminated, as relation is a set.

PROJECT Operator (π)

Syntax:

$\pi_{<\text{Attribute list}>}$ (Relation/Table name)

Properties:

1. The degree of output relation (number of columns present) is equal to the number of attributes mentioned in the attribute list.
2. Projection operator does not obey commutative property i.e.

$$\pi_{<list2>}(\pi_{<list1>}(R)) \neq \pi_{<list1>}(\pi_{<list2>}(R))$$

PROJECT Operator (π)

Examples: STUDENT

ROLL_NO	NAME	AGE	COURSE
1	ABHISHEK	17	BTech
2	AMIT	16	BCA
3	AJEET	17	BTech
4	AKHIL	18	BTech
5	PRASHANT	17	BCA

Query 1: Find out the name and course from STUDENT relation.

Syntax: $\pi_{\text{name, course}}$ (student)

Query 2: Find the age from STUDENT relation.

Syntax: π_{age} (student)

AGE
17
16
18

RENAME Operator (ρ)

- The results of relational algebra are also relations but without any name.
- The RENAME operation allows us to rename the relation.
- It is denoted by rho (ρ).

RENAME Operator (ρ)

Syntax:

ρ (Relation2, Relation1)

Examples:

Query 1: Rename STUDENT relation to STUDENT1 relation.

Syntax: ρ (STUDENT1, STUDENT)

Query 2: Create a relation STUDENT_NAMES with RNO and NAME from STUDENT.

Syntax: ρ (STUDENT_NAMES, $\pi_{(RNO, NAME)}$ (STUDENT))

Binary Relational Algebra Operations

3. Binary Relational Operations

I. JOIN (⋈)

Types of JOIN:

Various forms of join operation are:

A. Cross Join or Cartesian Product

B. Inner Joins:

(i). Theta join (ii). EQUI join (iii). Natural join

C. Outer join:

(i). Left Outer Join (ii). Right Outer Join

(iii). Full Outer Join

II. DIVISION (÷)

Join (⋈)

- Join is a binary operation which allows us to combine join product and selection in one single statement.

- The goal of creating a join condition is that it helps us to combine the data from two or more DBMS tables.

- Various forms of join operation are: Cross Join, Inner Join & Outer Join.

Cross Join or Cartesian Product (×)

Cross Join or Cartesian Product is symbolized by × symbol. It is an operation used to merge columns from two relations. It is also called Cross Product.

$R = A \times B$

Cross Join or Cartesian Product (×)

Exp.

Relation A

NAME	AGE
KAJAL	32
ANIL	40

Relation B

JOB	LOCATION
DEVELOPER	CHENNAI
ANALYST	MUMBAI

$R = A \times B$

NAME	AGE	JOB	LOCATION
KAJAL	32	DEVELOPER	CHENNAI
KAJAL	32	ANALYST	MUMBAI
ANIL	40	DEVELOPER	CHENNAI
ANIL	40	ANALYST	MUMBAI

Inner Join

- INNER JOIN is used to return rows from both tables which satisfy the given condition.
- It is the most widely used join operation and can be considered as a default join-type.
- Inner Join further divided into three subtypes:

(i). Theta join (ii). EQUI join (iii). Natural join

Inner Join: Theta Join

- THETA JOIN allows us to merge two tables based on the condition represented by theta.
- Theta joins work for all comparison operators.
- It is denoted by symbol θ.
- The general case of JOIN operation is called a Theta join.

Syntax: $\mathbf{A \bowtie_{\theta} B}$

R1 and R2 are relations having attributes (A_1, A_2, .., A_n) and (B_1, B_2, ..., B_n) such that the attributes don't have anything in common, that is R1 ∩ R2 = Φ.

Inner Join: Theta Join

Example:

STUDENT

SID	NAME	STD
1001	AJAY	11
1002	JATIN	12

SUBJECT

CLASS	SUBJECT
11	MATH
11	HINDI
12	COMPUTER
12	SCIENCE

$$\text{STUDENT} \bowtie_{\text{Student.Std} = \text{Subject.Class}} \text{SUBJECT}$$

Output:

SID	NAME	STD	CLASS	SUBJECT
1001	AJAY	11	11	MATH
1001	AJAY	11	11	HINDI
1002	JATIN	12	12	COMPUTER
1002	JATIN	12	12	SCIENCE

Inner Join: Equi Join

- EQUI JOIN is done when a Theta join uses only the equivalence condition.
- When Theta join uses only equality comparison operator, it is said to be equi join.
- The previous example corresponds to equi join.

Inner Join: Natural Join

- NATURAL JOIN does not utilize any of the comparison operators.
- In this type of join, the attributes should have the same name and domain.
- In Natural Join, there should be at least one common attribute between two relations.
- It performs selection forming equality on those attributes which appear in both relations and eliminates the duplicate attributes.

Inner Join: Natural Join

Example:

R1

NUM	SQUARE
2	4
3	9

R2

NUM	CUBE
2	8
3	27

R1 ⋈ R2

Output:

NUM	SQUARE	CUBE
2	4	8
3	9	27

Outer Join

- An OUTER JOIN doesn't require each record in the two join tables to have a matching record.
- In this type of join, the table retains each record even if no other matching record exists.
- Outer Join further divided into three subtypes:

    (i). Left Outer Join

    (ii). Right Outer Join

    (iii). Full Outer Join

Outer Join: Left Outer Join (⟕)

- LEFT OUTER JOIN returns all the rows from the table on the left even if no matching rows have been found in the table on the right.
- When no matching record found in the table on the right, NULL is returned.
- It is denoted by symbol (⟕).

Syntax: **A ⟕ B**

Outer Join: Left Outer Join (⟕)

Example:

R1

NUM	SQUARE
2	4
3	9
4	16

R2

NUM	CUBE
2	8
3	27
5	125

R1 ⟕ R2

Output:

NUM	SQUARE	CUBE
2	4	8
3	9	27
4	16	NULL

Outer Join: Right Outer Join (⟖)

- RIGHT OUTER JOIN returns all the columns from the table on the right even if no matching rows have been found in the table on the left.
- Where no matches have been found in the table on the left, NULL is returned. RIGHT OUTER JOIN is the opposite of LEFT OUTER JOIN.
- It is denoted by symbol (⟖).

Syntax: **A ⟖ B**

Outer Join: Right Outer Join (⟖)

Example:

R1

NUM	SQUARE
2	4
3	9
4	16

R2

NUM	CUBE
2	8
3	27
5	125

R1 ⟖ R2

Output:

NUM	CUBE	SQUARE
2	8	4
3	27	9
5	125	NULL

Outer Join: Full Outer Join (⟗)

- In a FULL OUTER JOIN , all tuples from both relations are included in the result, irrespective of the matching condition.
- It is denoted by symbol (⟗).

Syntax: **A ⟗ B**

Outer Join: Full Outer Join (⟗)

Example:

R1

NUM	SQUARE
2	4
3	9
4	16

R2

NUM	CUBE
2	8
3	27
5	125

R1 ⟗ R2

Output:

NUM	SQUARE	CUBE
2	4	8
3	9	27
4	16	NULL
5	NULL	125

DIVISION (÷)

Division operator A÷B can be applied if and only if:

- Attributes of B is proper subset of Attributes of A.
- The relation returned by division operator will have attributes = (All attributes of A – All Attributes of B).
- The relation returned by division operator will return those tuples from relation A which are associated to every B's tuple.

Syntax: **A ÷ B**

DIVISION (÷)

Example:

Relation A has 1 table as:

Sno	Pno
S1	P1
S1	P2
S1	P3
S1	P4
S1	P5
S1	P6
S2	P1
S2	P2
S3	P2
S4	P2
S4	P4
S4	P5

Relation B has 3 tables as:

Pno
P1

Pno
P2
P4

Pno
P1
P2
P3
P4
P5
P6

Then, A DIVIDE BY B gives the following resultant tables for all the three cases as follows:

Sno
S1
S2

Sno
S1
S4

Sno
S1

Exercise

Example 1: Consider the following schema:
SUPPLIER (Sid, S_name, S_addr)
PARTS (Pid, P_name, color)
CATALOG (sid, pid, cost)

Now, answer the following queries in Relational Algebra:
(a) Find the name of all the suppliers who supply Yellow parts.
(b) Find the Sid of suppliers who supply every parts.
(c) Find the Sid of suppliers who supply every red or green part.

Exercise

Solution:

(a) Find the name of all the suppliers who supply Yellow parts.

$$\pi_{S_name}(SUPPLIER) \bowtie CATALOG \bowtie (\sigma_{color = \text{'Yellow'}}(PARTS))$$

(b) Find the Sid of suppliers who supply every parts.

$$\pi_{sid,\ pid}(CATALOG) \div \pi_{pid}(PARTS)$$

(c) Find the Sid of suppliers who supply every red or green part.

$$(\pi_{sid,\ pid}(CATALOG)) \div (\pi_{pid}(\sigma_{color = \text{'red'} \vee color = \text{'green'}}(PARTS)))$$

Exercise

Example 2: Consider a database that has the relation schema CR (StudentName, CourseName). An instance of the schema CR is as given below.

StudentName	CourseName
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

The following queries are made on the database.

$T1 \leftarrow \pi_{CourseName}(\sigma_{StudentName='SA'}(CR))$

$T2 \leftarrow CR \div T1$

The number of rows in T2 are _____.

(GATE 2017)

Exercise

Solution:

T1 will give:

CourseName
CA
CB
CC

T2←CR÷T1 = All the tuples in CR which are matched with every tuple in T1:

StudentName
SA
SC
SD
SF

Reference Books

1. Elmasri and Navathe (2010), “Fudamentals of Database Systems”, 5th Edition, Addision Wesley.
2. Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
3. Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
4. M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.

Thank You !!!

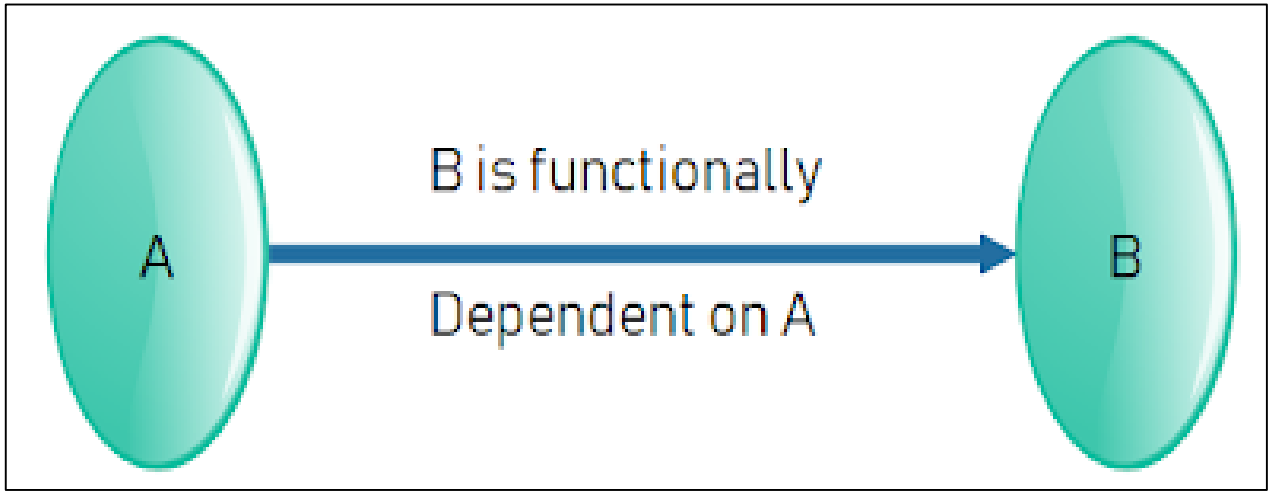
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management Systems (BCSC – 1003)

Topic: Functional Dependency



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Functional Dependency

- The functional dependency is a relationship that exists between two attributes.
- It typically exists between the primary key and non-key attribute within a table.
- It is usually denoted as: X → Y
- The left side of arrow is known as a determinant, the right side of the arrow is known as a dependent.

Functional Dependency

Example:

Assume, we have an EMPLOYEE table with attributes: Emp_Id, Emp_Name, Emp_Address.

Here Emp_Id attribute can uniquely identify the Emp_Name attribute of employee table because if we know the Emp_Id, we can tell that employee name associated with it. Functional dependency can be written as:

$$Emp_Id \rightarrow Emp_Name$$

We can say that Emp_Name is functionally dependent on Emp_Id.

Functional Dependency

Types of Functional Dependencies:

1. Trivial functional dependency
2. Non-trivial functional dependency
3. Multivalued dependency
4. Transitive dependency

Functional Dependency

1. Trivial functional dependency

A → B has trivial functional dependency if B is a subset of A. The following dependencies are also trivial like: A → A, B → B

Example:

Consider a table with two columns Employee_Id and Employee_Name.

{Employee_id, Employee_Name} → Employee_Id is a trivial functional dependency as Employee_Id is a subset of {Employee_Id, Employee_Name}.

Also, Employee_Id → Employee_Id and

Employee_Name → Employee_Name are trivial dependencies too.

Functional Dependency

2. Non-trivial functional dependency

A → B has a non-trivial functional dependency if B is not a subset of A.

When A intersection B is NULL, then A → B is called as complete non-trivial.

Example:

ID → Name,

Name → DOB

Functional Dependency

3. Multivalued functional dependency

When existence of one or more rows in a table implies one or more other rows in the same table, then the Multi-valued dependencies occur.

If a table has attributes P, Q and R, then Q and R are multi-valued facts of P. It is represented by double arrow as $\rightarrow\rightarrow$

Example 1:

$P \rightarrow\rightarrow Q$

$P \rightarrow\rightarrow R$

In this case, Multivalued Dependency exists only if Q and R are independent attributes.

Functional Dependency

Example: Consider following table Prog_Proj below showing Multivalued dependency.

PROG_ PROJ

Programmer	Project	Modules
Vineet	P3	M61
Nikhil	P1	M31
Sanjay	P6	M99
Nikhil	P2	M61
Arvind	P1	M91
Nikhil	P1	M71
Sanjay	P7	M91
Nikhil	P2	M75
Vijay	P9	M61

Here columns PROJECT & MODULES are independent of each other and dependent on PROGRAMMER. In this case these two columns are said to be multivalued dependent on PROGRAMMER. These dependencies can be represented like this:

PROGRAMMER → → PROJECT and PROGRAMMER → → MODULES

Functional Dependency

4. Transitive functional dependency

A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies.

Example:

X → Z is a transitive dependency if the following functional dependencies hold true:

- X → Y
- Y → Z

A transitive dependency can only occur in a relation of three of more attributes. This dependency helps us normalizing the database in 3NF (3rd Normal Form).

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you

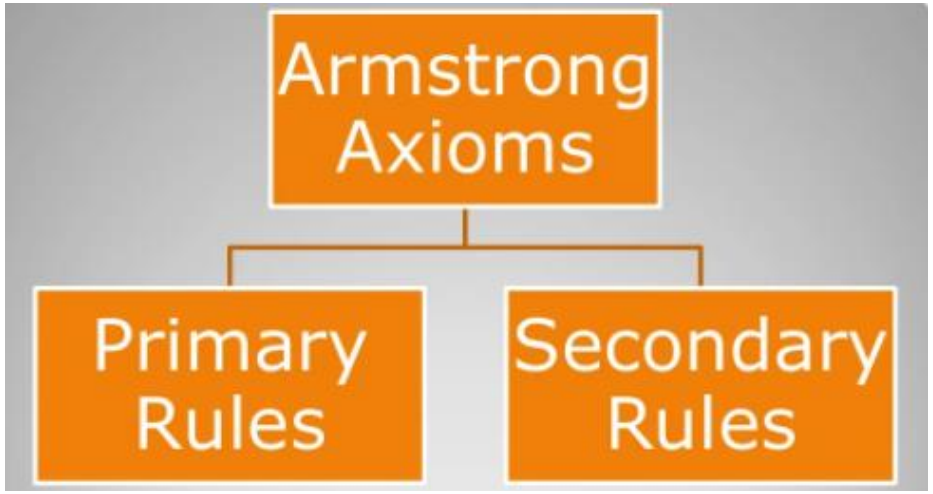
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC–1003)

Topic: Armstrong Axioms (Inference Rules)



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

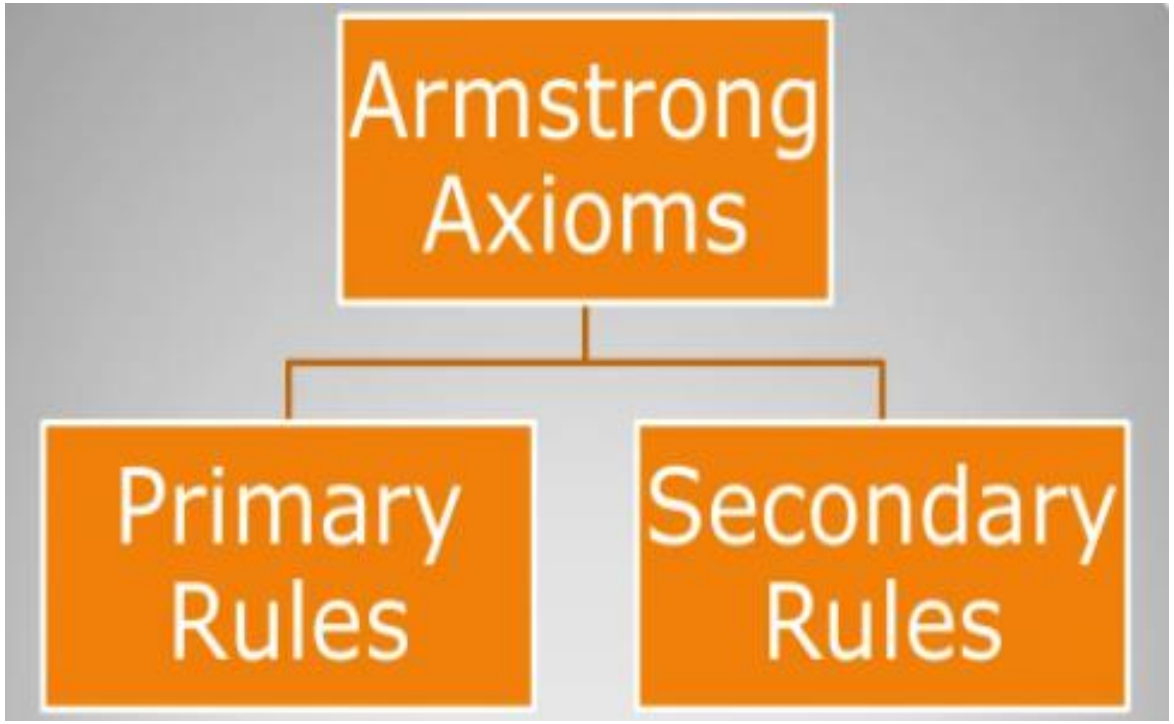
Armstrong Axioms (Inference Rule) in Functional Dependecy in DBMS

Introduction to Axioms Rules:

- Armstrong's Axioms is a set of rules.
- It provides a simple technique for reasoning about functional dependencies.
- William W. Armstrong developed it in 1974.
- It is used to infer all the functional dependencies on a relational database.

Armstrong Axioms (Inference Rule)

Armstrong Axioms can be classified as:



Armstrong Axioms (Inference Rule)

Consider a relation **STUDENT**

RNo	Name	Marks	Dept	Course
1	A	78	CS	C1
2	B	60	EE	C1
3	A	78	CS	C2
4	B	60	EE	C3
5	C	80	IT	C2

Primary Rules

Rule 1. Reflexivity

- $\{A \rightarrow A\}$ and,
- If A is a set of attributes and B is a subset of A, then A holds B. $\{A \rightarrow B\}$. This is trivial property.

Example:

RNo → RNo (✓)

(RNo, Name) → Name (✓)

Primary Rules

Rule 2. Augmentation

- If A → B holds and Y is attribute set, then AY → BY also holds.
- That means adding attributes in dependencies, does not change the basic dependencies.
- If A → B, then AC → BC for any C.

Example:

If RNo → Name (✓)

then {RNo, Marks} → {Name, Marks} (✓)

Primary Rules

Rule 3. Transitivity

- If A holds B and B holds C, then A holds C.
- It means if $\{A \rightarrow B\}$ and $\{B \rightarrow C\}$, then $\{A \rightarrow C\}$.
- A holds B $\{A \rightarrow B\}$ means that A functionally determines B.

Example:

If Name → Marks (✓) and Marks → Dept (✓)

then Name → Dept (✓)

Secondary Rules

Rule 1. Union

- If $\{A \rightarrow B\}$ and $\{A \rightarrow C\}$, then $\{A \rightarrow BC\}$

Example:

If RNo → Name (✓) and RNo → Marks (✓)

then RNo → Name, Marks (✓)

Secondary Rules

Rule 2. Decomposition or Splitting property

- If {A → BC} then {A → B} and {A → C}
- *But if AB → C then we can't split the LHS as A → C and B → C*

Example:

If (RNo → Name, Marks) (✓)

Then (RNo → Name) (✓) and (RNo → Marks) (✓)

Secondary Rules

Rule 3. Pseudo Transitivity

- If{A → B} and {BC → D}, then {AC → D}

Example:

RNo → Name (✓) and (Name, Marks) → Dept (✓)

then (RNo, Marks) → Dept (✓)

Secondary Rules

Rule 4. Composition

- If X → Y and A → B then XA → YB

Example:

RNo → Name (✓) and Marks → Dept (✓)

then (RNo, Marks) → (Name, Dept) (✓)

Example

Question: Consider relation E = (P, Q, R, S, T, U) having set of following Functional Dependencies (FD).

$P \rightarrow Q$	$P \rightarrow R$	$QR \rightarrow S$
$Q \rightarrow T$	$QR \rightarrow U$	$PR \rightarrow U$

Calculate some members of Axioms are as follows:

1. $P \rightarrow T$
2. $PR \rightarrow S$
3. $QR \rightarrow SU$
4. $PR \rightarrow SU$

Example

Solution:

1. $P \rightarrow T$

In the above FD set, $P \rightarrow Q$ and $Q \rightarrow T$

So, Using Transitive Rule: If $\{A \rightarrow B\}$ and $\{B \rightarrow C\}$, then $\{A \rightarrow C\}$

$\therefore$ If $P \rightarrow Q$ and $Q \rightarrow T$, then $P \rightarrow T$.

$P \rightarrow T$

Example

2. PR → S

In the above FD set, P → Q

As, QR → S

So, Using Pseudo Transitivity Rule: If{A → B} and {BC → D}, then {AC → D}

∴ If P → Q and QR → S, then PR → S.

PR → S

Example

3. QR → SU

In above FD set, QR → S and QR → U

So, Using Union Rule: If{A → B} and {A → C}, then {A → BC}

∴ If QR → S and QR → U, then QR → SU.

QR → SU

Example

4. PR → SU

So, Using Union Rule: If{A → B} and {A → C}, then {A → BC}

∴ If PR → S and PR → U, then PR → SU.

PR → SU

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you

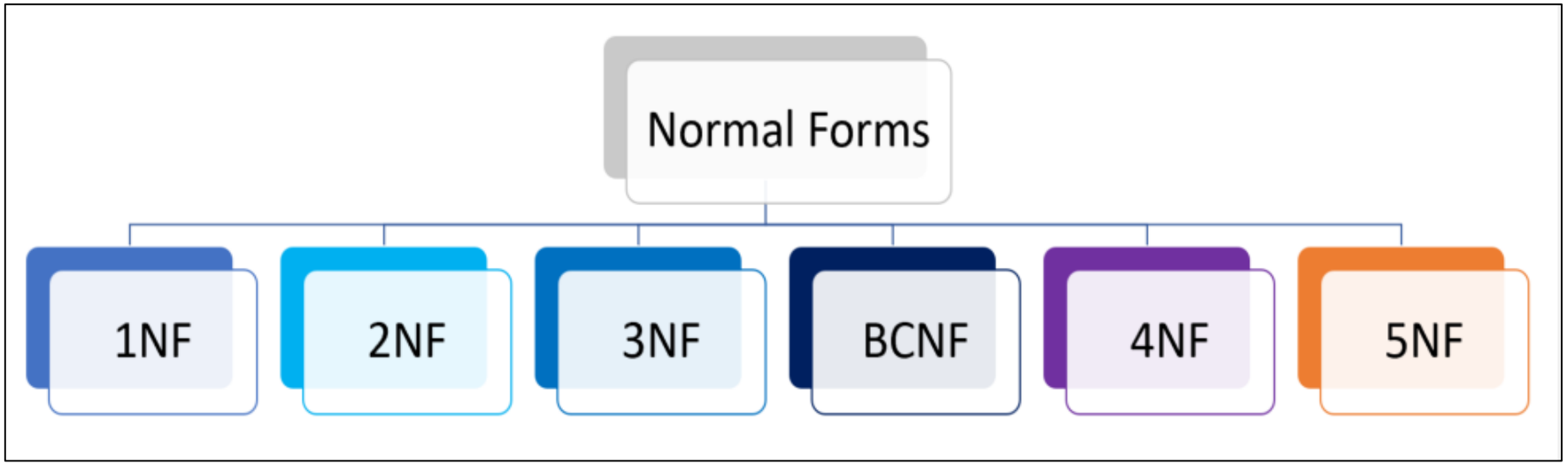
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management Systems (BCSC-0003)

Topic: Normalization



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Normalization

- Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.

- Normalization rules divides larger tables into smaller tables and links them using relationships.

- The purpose of Normalization in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.

Anomalies

Anomalies are problems that can occur in poorly planned, un-normalized databases where all the data is stored in one table.

Types of Anomalies:

1. Insert Anomaly,
2. Update Anomaly, and
3. Delete Anomaly.

Anomalies

- Insertion Anomaly:

An Insert Anomaly occurs when certain attributes cannot be inserted into the database without the presence of other attributes.

- Update or Modification Anomaly:

An update anomaly is a data inconsistency that results from data redundancy and a partial update.

- Deletion Anomaly:

Deletion Anomaly. A deletion anomaly occurs when you delete a record that may contain attributes that shouldn't be deleted.

Anomalies

SRNO	SNAME	CREDIT	DEPT_NAME	BUILDING	ROOM_NO
1	Rahul	5	CSE	B1	101
2	Jitendra	8	CSE	B1	101
3	Jagdish	9	EE	B2	201
4	Payal	7	EE	B2	201
5	Ankur	9	CIVIL	B1	110
6	Akash	8	ECE	B1	115
7	Vansh	7	CIVIL	B1	110
8	Tanuj	6	CSE	B1	101

Insert Anomalies

SRNO	SNAME	CREDIT	DEPT_NAME	BUILDING	ROOM_NO
1	Rahul	5	CSE	B1	101
2	Jitendra	8	CSE	B1	101
3	Jagdish	9	EE	B2	201
4	Payal	7	EE	B2	201
5	Ankur	9	CIVIL	B1	110
6	Akash	8	ECE	B1	115
7	Vansh	7	CIVIL	B1	110
8	Tanuj	6	CSE	B1	101
-	-	-	ME	B1	120



Update Anomalies

SRNO	SNAME	CREDIT	DEPT_NAME	BUILDING	ROOM_NO
1	Rahul	5	CSE	~~B1~~ B4	~~101~~ 301
2	Jitendra	8	CSE	~~B1~~ B4	~~101~~ 301
3	Jagdish	9	EE	B2	201
4	Payal	7	EE	B2	201
5	Ankur	9	CIVIL	B1	110
6	Akash	8	ECE	B3	115
7	Vansh	7	CIVIL	B1	110
8	Tanuj	6	CSE	~~B1~~ B4	~~101~~ 301

Delete Anomalies

SRNO	SNAME	CREDIT	DEPT_NAME	BUILDING	ROOM_NO
1	Rahul	5	CSE	B1	101
2	Jitendra	8	CSE	B1	101
3	Jagdish	9	EE	B2	201
4	Payal	7	EE	B2	201
5	Ankur	9	CIVIL	B1	110
6	Akash	8	ECE	B1	115
7	Vansh	7	CIVIL	B1	110
8	Tanuj	6	CSE	B1	101

Solution

STUDENT

SRNO	SNAME	CREDIT	DEPT_NAME
1	Rahul	5	CSE
2	Jitendra	8	CSE
3	Jagdish	9	EE
4	Payal	7	EE
5	Ankur	9	CIVIL
6	Akash	8	ECE
7	Vansh	7	CIVIL
8	Tanuj	6	CSE

DEPARTMENT

DEPT_NAME	BUILDING	ROOM_NO
CSE	B4	301
EE	B2	201
CIVIL	B1	110
ECE	B1	115

Advantages of Normalization

Following are some of the advantages of normalization:

1.To reduce the redundancy from the table.

2. To save the memory space as data can be stored in compact form.

3. To remove Insert, Update and Delete anomalies.

4. Normalization minimizes the null values.

5. Using normalization, we can simple the queries.

6. It is important for OLTP (Online Transaction Processing). However De-normalization supports OLAP (Online Analytical Processing).

7. It simplifies the database structure.

8. Searching, sorting and creating indexes will be faster after applying the normalization.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you

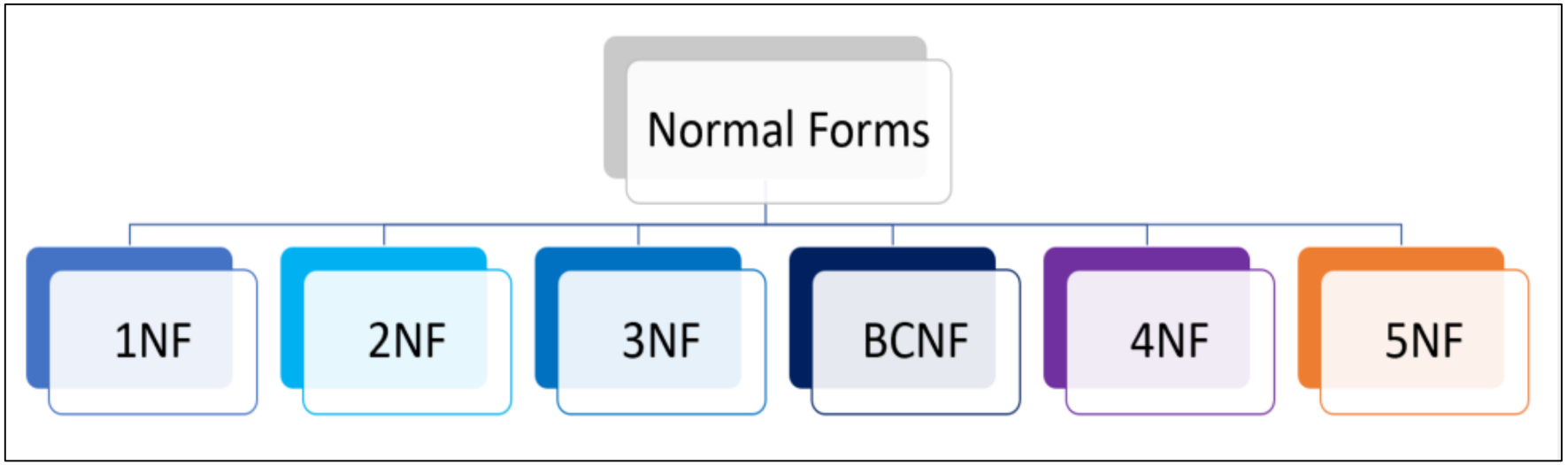
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management Systems (BCSC-0003)

Topic: Normalization



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Normalization

- Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.

- Normalization rules divides larger tables into smaller tables and links them using relationships.

- The purpose of Normalization in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.

Types of Normal Form

Here is a list of Normal Forms

- 1NF (First Normal Form)
- 2NF (Second Normal Form)
- 3NF (Third Normal Form)
- BCNF (Boyce-Codd Normal Form)
- 4NF (Fourth Normal Form)
- 5NF (Fifth Normal Form)
- 6NF (Sixth Normal Form)

First Normal Form (1NF)

- A relation will be in 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

First Normal Form (1NF)

Example:

- Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_PHONE	STATE
2925	RAJESH	9500407832, 9088170884	TAMILNADU
6796	KISHORE	9510508668	KARNATAKA
4592	ALOK	8455661732	PUNJAB
3565	HEMANT	9759105522, 9414525254	GUJRAT

First Normal Form (1NF)

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	STATE
2925	RAJESH	9500407832	TAMILNADU
2925	RAJESH	9088170884	TAMILNADU
6796	KISHORE	9510508668	KARNATAKA
4592	ALOK	8455661732	PUNJAB
3565	HEMANT	9759105522	GUJRAT
3565	HEMANT	9414525254	GUJRAT

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key.

Example:

Let's assume, a college can store the data of teachers and the subjects they teach. In a college, a teacher can teach more than one subject.

TEACHER table:

TEACHER_ID	SUBJECT	TEACHER_AGE
2529	Information Technology	30
2529	Soft Computing	30
6535	Data Communication	45
3578	Soft Computing	35
3578	Digital Image Processing	35

Second Normal Form (2NF)

- In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID. That's why it violates the rule for 2NF.
- To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL Table and TEACHER_SUBJECT Table:

TEACHER_ID	TEACHER_AGE
2529	30
6535	45
3578	35

TEACHER_ID	SUBJECT
2529	Information Technology
2529	Soft Computing
6535	Data Communication
3578	Soft Computing
3578	Digital Image Processing

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds at least one of the following conditions for every non-trivial function dependency X → Y.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Third Normal Form (3NF)

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}.... so on.

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key (EMP_ID). It violates the rule of third normal form.

Third Normal Form (3NF)

- That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP Table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

Boyce-Codd Normal Form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

Boyce-Codd Normal Form (BCNF)

In the above table Functional dependencies are as follows:

- EMP_ID → EMP_COUNTRY
- EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.


Thank
you

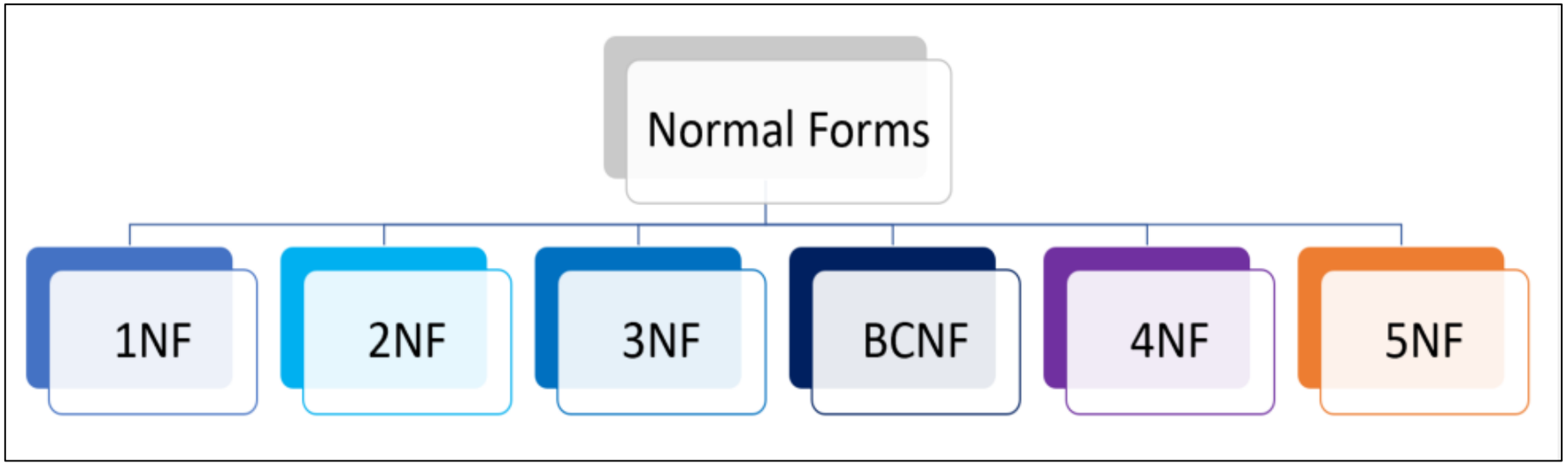
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management Systems (BCSC-0003)

Topic: Normal Forms: 4NF & 5NF



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Fourth Normal Form (4NF)

- A relation will be in 4NF if it is in Boyce–Codd normal form and has no multi-valued dependency.
- For a dependency A → B, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Example: STUDENT

STD_ID	COURSE	HOBBY
21	COMPUTER	DANCING
21	MATH	SINGING
34	CHEMISTRY	DANCING
74	BIOLOGY	CRICKET
59	PHYSICS	HOCKEY

Fourth Normal Form (4NF)

- The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity.
- Hence, there is no relationship between COURSE and HOBBY.
- In the STUDENT relation, a student with STD_ID, 21 contains two courses, Computer and Math and two hobbies, Dancing and Singing.
- So there is a Multi-valued dependency on STD_ID, which leads to unnecessary repetition of data.

Fourth Normal Form (4NF)

- So to make the above table into 4NF, we can decompose it into two tables:

Example: STUDENT_COURSE

STD_ID	COURSE
21	COMPUTER
21	MATH
34	CHEMISTRY
74	BIOLOGY
59	PHYSICS

STUDENT_HOBBY

STD_ID	HOBBY
21	DANCING
21	SINGING
34	DANCING
74	CRICKET
59	HOCKEY

Fifth Normal Form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

Example: SUBJECT_DETAILS

SUBJECT	LECTURER	SEMESTER
COMPUTER	ANSHIKA	SEMESTER 1
COMPUTER	JOHN	SEMESTER 1
MATH	JOHN	SEMESTER 1
MATH	AKASH	SEMESTER 2
CHEMISTRY	PRAVEEN	SEMESTER 1

Fifth Normal Form (5NF)

- In the SUBJECT_DETAILS table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data.

- Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

- So to make the above table into 5NF, we can decompose it into three relations T1, T2 & T3:

Fifth Normal Form (5NF)

T1 table:

SEMESTER	SUBJECT
SEMESTER 1	COMPUTER
SEMESTER 1	MATH
SEMESTER 1	CHEMISTRY
SEMESTER 2	MATH

T2 table:

SUBJECT	LECTURER
COMPUTER	ANSHIKA
COMPUTER	JOHN
MATH	JOHN
MATH	AKASH
CHEMISTRY	PRAVEEN

Fifth Normal Form (5NF)

T3 table:

SEMESTER	LECTURER
SEMESTER 1	ANSHIKA
SEMESTER 1	JOHN
SEMESTER 1	JOHN
SEMESTER 2	AKASH
SEMESTER 1	PRAVEEN

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/ last accessed on 07 Feb’2021.


Thank
you

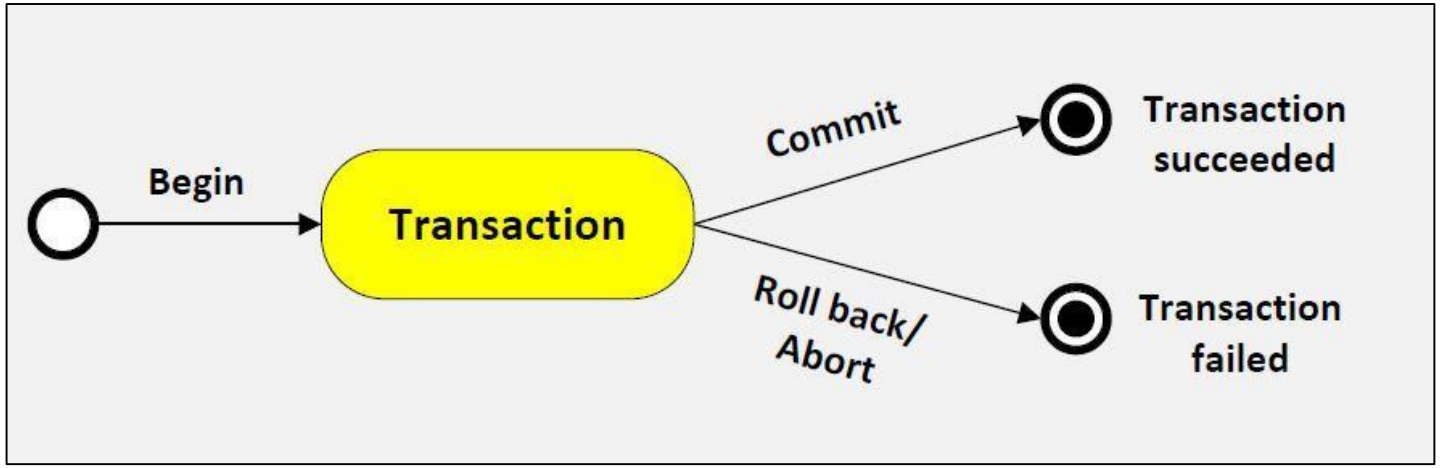
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC – 1003)

Topic: Transaction Processing



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- Transaction
- Operations of Transaction
- Transaction Property
- State of Transaction

Transaction

- The transaction is a set of logically related operation. It contains a group of tasks.
- A transaction is an action or series of actions. It is performed usually by a single user to perform operations for accessing the contents of the database.

Example: Suppose an employee of bank transfers Rs 800 from X's account to Y's account. This small transaction contains several low-level tasks:

X’s A/c

```
Open_Account(X)
Old_Balance = X.balance
New_Balance = Old_Balance - 800
X.balance = New_Balance
Close_Account(X)
```

Y’s A/c

```
Open_Account(Y)
Old_Balance = Y.balance
New_Balance = Old_Balance + 800
Y.balance = New_Balance
Close_Account(Y)
```

Operations of Transaction

Following are the main operations of transaction:

- Read (X): Read operation is used to read the value of X from the database and stores it in a buffer in main memory.
- Write (X): Write operation is used to write the value back to the database from the buffer.

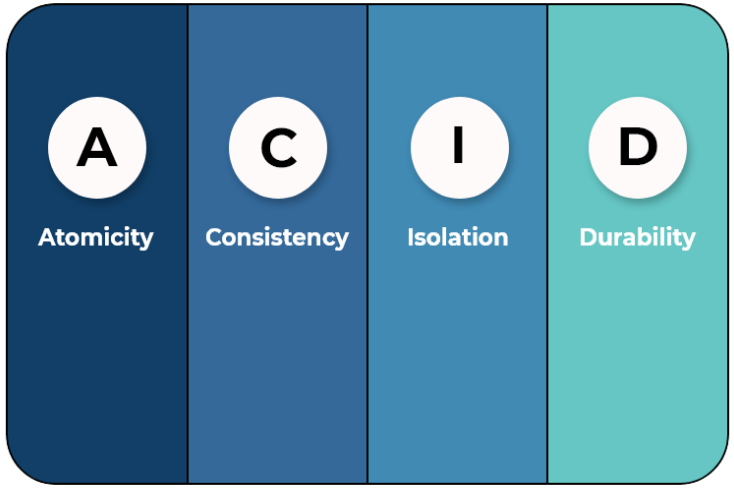
- Commit: It is used to save the work done permanently.
- Rollback: It is used to undo the work done.

Transaction Property

- The transaction has the four properties.

- These are used to maintain consistency in a database, before and after the transaction.

- Property of Transaction are:
  - ❖ Atomicity
  - ❖ Consistency
  - ❖ Isolation
  - ❖ Durability



Atomicity

- It states that all operations of the transaction take place at once if not, the transaction is aborted.
- There is no midway, i.e., the transaction cannot occur partially.
- Each transaction is treated as one unit and either run to completion or is not executed at all.
- Atomicity involves the following two operations:
  - ❖ Abort: If a transaction aborts then all the changes made are not visible.
  - ❖ Commit: If a transaction commits then all the changes made are visible.

Consistency

- The integrity constraints are maintained so that the database is consistent before and after the transaction.
- The execution of a transaction will leave a database in either its prior stable state or a new stable state.
- The transaction is used to transform the database from one consistent state to another consistent state.

Isolation

- It shows that the data which is used at the time of execution of a transaction cannot be used by the second transaction until the first one is completed.

- In isolation, if the transaction T1 is being executed and using the data item X, then that data item can't be accessed by any other transaction T2 until the transaction T1 ends.

- The concurrency control subsystem of the DBMS enforced the isolation property.

Durability

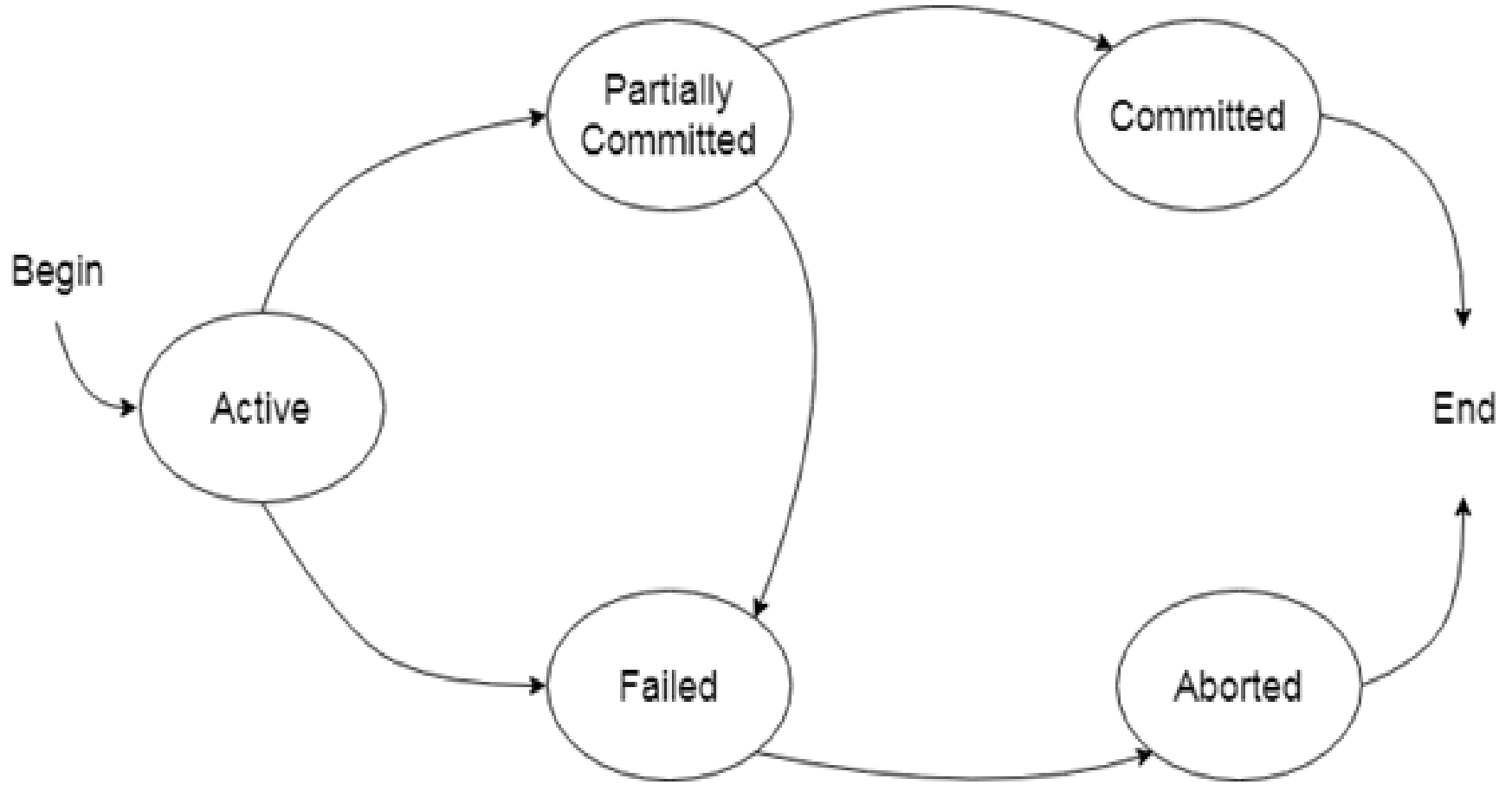
- The durability property is used to indicate the performance of the database's consistent state. It states that the transaction made the permanent changes.

- They cannot be lost by the system failure. When a transaction is completed, then the database reaches a state known as the consistent state. That consistent state cannot be lost, even in the event of a system's failure.

- The recovery subsystem of the DBMS has the responsibility of Durability property.

State of Transaction

In a database, the transaction can be in one of the following states:



State of Transaction

Active state:

- The active state is the first state of every transaction. In this state, the transaction is being executed.
- For example: Insertion or deletion or updating a record is done here. But all the records are still not saved to the database.

Partially committed:

- In the partially committed state, a transaction executes its final operation, but the data is still not saved to the database.
- In the total mark calculation example, a final display of the total marks step is executed in this state.

State of Transaction

Committed:

- A transaction is said to be in a committed state if it executes all its operations successfully.
- In this state, all the effects are now permanently saved on the database system.

Failed state:

- If any of the checks made by the database recovery system fails, then the transaction is said to be in the failed state.
- In the example of total mark calculation, if the database is not able to fire a query to fetch the marks, then the transaction will fail to execute.

State of Transaction

Aborted:

- If any of the checks fail and the transaction has reached a failed state then the database recovery system will make sure that the database is in its previous consistent state. If not then it will abort or roll back the transaction to bring the database into a consistent state.
- If the transaction fails in the middle of the transaction then before executing the transaction, all the executed transactions are rolled back to its consistent state.
- After aborting the transaction, the database recovery module will select one of the two operations:
  - ❖ Re-start the transaction
  - ❖ Kill the transaction

References

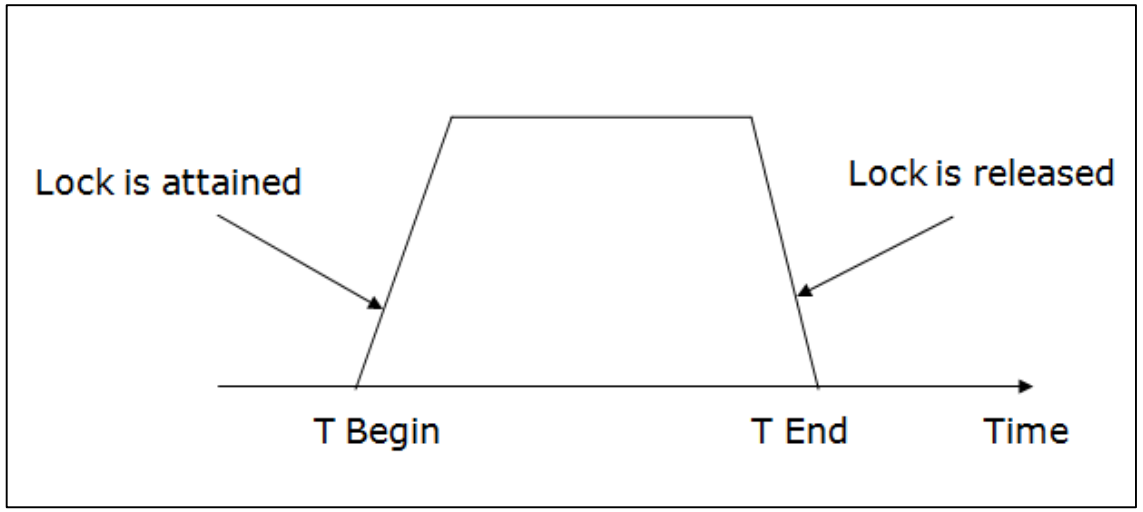
- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-file-organization/ last accessed on 08 September' 2021.


Thank
you

GLA UNIVERSITY MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A Grade by NAAC
12-B Status from UGC

Database Management System (BCSC – 1003)

Topic: Concurrency Control Techniques



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- Concurrent execution in DBMS
- Problems with concurrent execution
- Concurrency Control Protocol
  - ❖Lock Based Protocol
  - ❖Timestamp Concurrency Control Protocol
  - ❖Validation Based Concurrency Control Protocol

Concurrent Execution in DBMS

- In a multi-user system, multiple users can access and use the same database at one time, which is known as the concurrent execution of the database.

- While working on the database transactions, there occurs the requirement of using the database by multiple users for performing different operations, and in that case, concurrent execution of the database is performed.

- The simultaneous execution that is performed should be done in an concurrent manner, and no operation should affect the other executing operations, thus maintaining the consistency of the database.

Problems with Concurrent Execution

In database transaction, READ & WRITE are two main operations. So, there is a need to manage these operations in the concurrent execution of the transactions. Otherwise data may be inconsistent & have following problems:

- Lost Update Problems: The problem occurs when two different database transactions perform the read/write operations on the same database items and update the values of the items incorrect.

- Dirty Read Problems: The dirty read problem occurs when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction.

Concurrency Control Protocols

- Concurrency Control is the working concept that is required for controlling and managing the concurrent execution of database operations and thus avoiding the inconsistencies in the database. Thus, for maintaining the concurrency of the database, we have the concurrency control protocols.

- The concurrency control protocols ensure the atomicity, consistency, isolation, durability and serializability of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:

  - ❖ Lock Based Concurrency Control Protocol
  - ❖ Time Stamp Concurrency Control Protocol
  - ❖ Validation Based Concurrency Control Protocol

Lock-Based Protocol

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:

1. Shared lock: Shared lock is also called read lock, used for reading data items only. They ensure that a record is not in process of being updated during a read-only request. Shared locks can also be used to prevent any kind of updates of record. It is denoted by Lock-S.

2. Exclusive lock: With the Exclusive Lock, a data item can be read as well as written. Also called write lock. An exclusive lock prevents any other locker from obtaining any sort of a lock on the object. They can be owned by only one transaction at a time. It is denoted as Lock-X.

Two – Phase Locking (2PL)

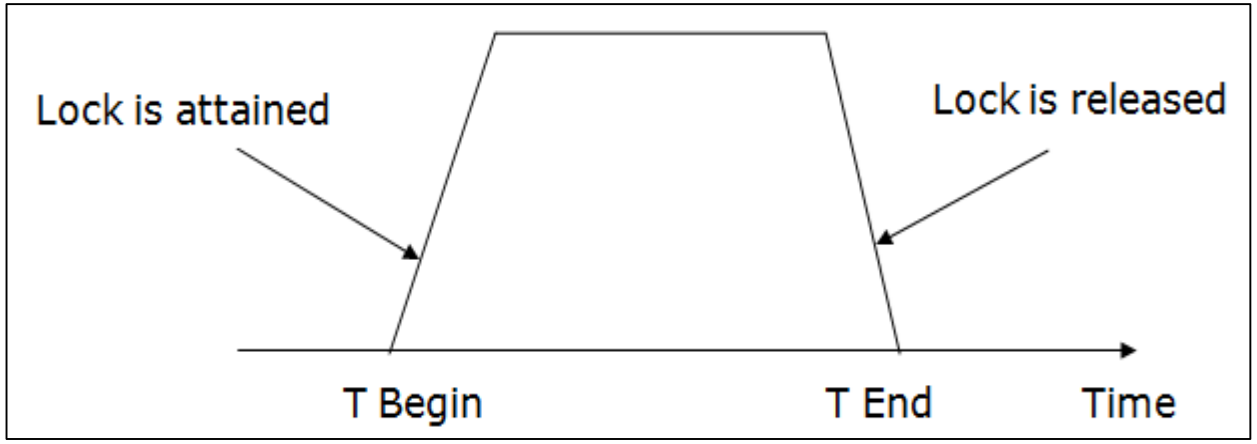
- The two-phase locking protocol divides the execution phase of the transaction into three parts.

- In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires.

- In the second part, the transaction acquires all the locks.

- The third phase is started as soon as the transaction releases its first lock. In this phase, the transaction cannot demand any new locks. It only releases the acquired locks.

Two – Phase Locking (2PL)



There are two phases of 2PL:

- Growing phase: In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.

- Shrinking phase: In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired.

Timestamp Ordering Protocol

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps.
- The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first.
- To determine the timestamp of the transaction, this protocol uses system time.

Validation Based Protocol

It is also known as optimistic concurrency control technique. In the validation based protocol, the transaction is executed in the following three phases:

1. Read phase: In this phase, the transaction T is read and executed. It is used to read the value of various data items and stores them in temporary local variables. It can perform all the write operations on temporary variables without an update to the actual database.

2. Validation phase: In this phase, the temporary variable value will be validated against the actual data to see if it violates the serializability.

3. Write phase: If the validation of the transaction is validated, then the temporary results are written to the database otherwise the transaction is rolled back.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-concurrency-control/ last accessed on 8 October' 2021.


Thank
you

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC – 0003)

Topic: Log Based Recovery



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- Log Based Recovery
- Log Record Representation
- Types of Log Records
- Approaches to Modify Database

Log Based Recovery

- The log is a sequence of records.
- Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.
- Any operation which is performed on the database is recorded on the log.
- Prior to performing any modification to database, an update log record is created to reflect that modification.

Log Record Representation

- An update log record represented as: <Ti, Xj, V_1, V_2> has these fields:

1. Transaction identifier: Unique Identifier of the transaction that performed the write operation.
2. Data item: Unique identifier of the data item written.
3. Old value: Value of data item prior to write.
4. New value: Value of data item after write operation.

Types of Log Records

Other type of log records are:

- <Ti start>: It contains information about when a transaction Ti starts.
- <Ti commit>: It contains information about when a transaction Ti commits.
- <Ti abort>: It contains information about when a transaction Ti aborts.

Approaches to Modify Database

There are two approaches to modify the database:

1. Deferred database modification: The deferred modification technique occurs if the transaction does not modify the database until it has committed. In this method, all the logs are created and stored in the stable storage, and the database is updated when a transaction commits.

2. Immediate database modification: The Immediate modification technique occurs if database modification occurs while the transaction is still active. In this technique, the database is modified immediately after every operation.

References

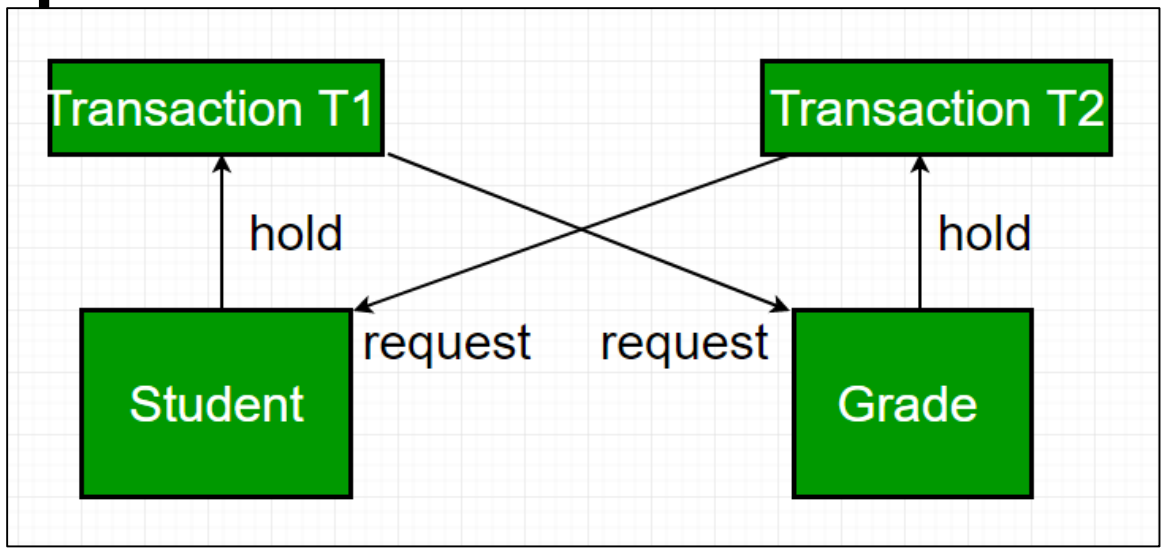
- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-log-based-recovery/ last accessed on 12 September' 2021.


Thank
you

GLA UNIVERSITY MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A Grade by NAAC
12-B Status from UGC

Database Management System (BCSC – 0003)

Topic: Deadlock in DBMS



Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- Deadlock in DBMS
- Deadlock Avoidance
- Deadlock Detection
- Deadlock Prevention

Deadlock in DBMS

- In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for one another to give up locks.

- Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a Halt.

Deadlock in DBMS

For example:

In the STUDENT table, transaction T1 holds a lock on some rows and needs to update some rows in the GRADE table. Simultaneously, transaction T2 holds locks on some rows in the GRADE table and needs to update the rows in the STUDENT table held by Transaction T1.

Now, the main problem arises. Transaction T1 is waiting for T2 to release its lock and similarly, transaction T2 is waiting for T1 to release its lock. All activities come to a halt state and remain at a standstill. It will remain in a standstill until the DBMS detects the deadlock and aborts one of the transactions.

Deadlock in DBMS

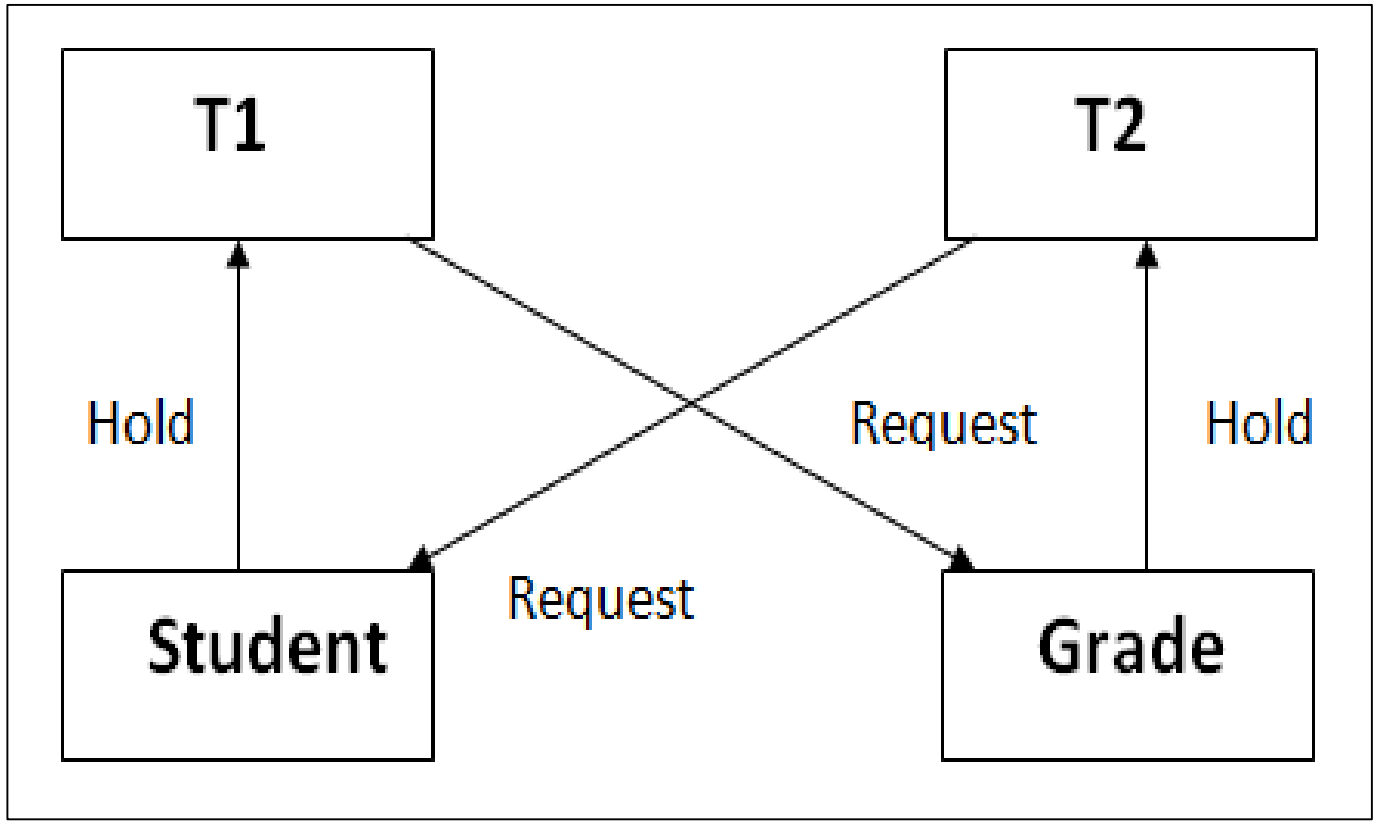


Figure: Deadlock in DBMS

Deadlock Avoidance

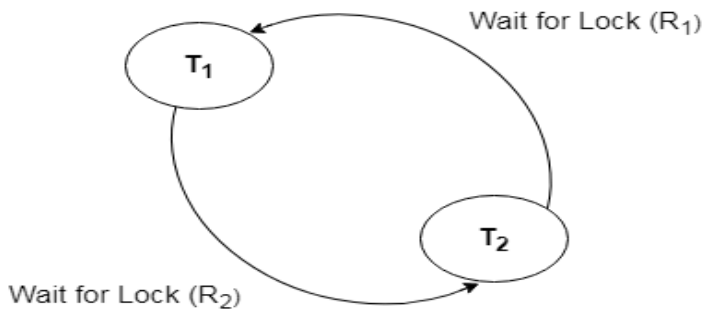
- When a database is stuck in a deadlock, It is always better to avoid the deadlock rather than restarting or aborting the database.

- Deadlock avoidance method is suitable for smaller databases whereas deadlock prevention method is suitable for larger databases.

Deadlock Detection

- In a database, when a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is involved in a deadlock or not.

Wait-for-graph is one of the methods for detecting the deadlock situation. This method is suitable for smaller database. In this method a graph is drawn based on the transaction and their lock on the resource. If the graph created has a closed loop or a cycle, then there is a deadlock.



Deadlock Prevention

- Deadlock prevention method is suitable for a large database. If the resources are allocated in such a way that deadlock never occurs, then the deadlock can be prevented.

- The DBMS analyzes the operations of the transaction whether they can create a deadlock situation or not. If they do, then the DBMS never allowed that transaction to be executed.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/deadlock-in-dbms/ last accessed on 28 September' 2021.


Thank
you

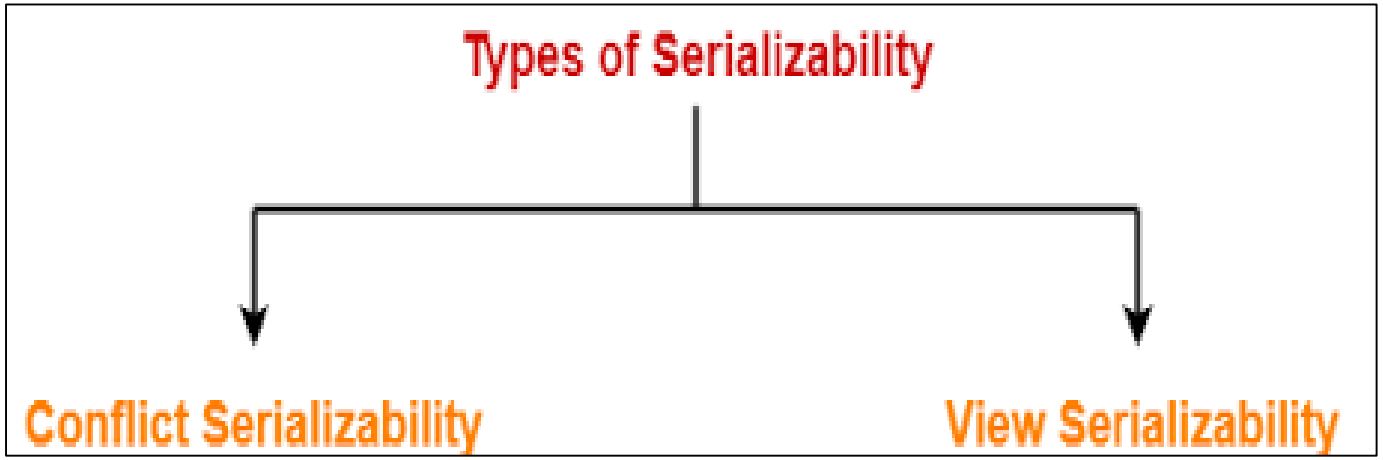
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC – 1003)

Topic: DBMS Serializability



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- Serializability
- Serializability Schedule
- Types of Serializability
- Recoverability of Schedule

Serializability

- When multiple transactions are running concurrently then there is a possibility that the database may be left in an inconsistent state.

- Serializability is a concept that helps us to check which schedules are serializable.

- A serializable schedule is the one that always leaves the database in consistent state.

Serializable Schedule

- A serializable schedule always leaves the database in consistent state.
- A serial schedule is always a serializable schedule because in serial schedule, a transaction only starts when the other transaction finished execution.
- However a non-serial schedule needs to be checked for Serializability.
- A serial schedule doesn’t allow concurrency, only one transaction executes at a time and the other starts when the already running transaction finished.

Types of Serializability

There are two types of Serializability.

1. Conflict Serializability &

2. View Serializability

Conflict Serializability

- A schedule is called conflict serializable if we can convert it into a serial schedule after swapping its non-conflicting operations.

Conflicting operations:

- Two operations are said to be in conflict, if they satisfy all the following three conditions:

1. Both the operations should belong to different transactions.
2. Both the operations are working on same data item.
3. At least one of the operation is a write operation.

Conflict Serializability

Lets see some examples to understand this:

Exp 1: Operation W(X) of transaction T1 and operation R(X) of transaction T2 are conflicting operations, because they satisfy all the three conditions mentioned above. They belong to different transactions, they are working on same data item X, one of the operation in write operation.

Exp 2: Operations W(X) of T1 & W(X) of T2 are conflicting operations.

Exp 3: Operations W(X) of T1 and W(Y) of T2 are non-conflicting operations because both the write operations are not working on same data item so these operations don’t satisfy the second condition.

Exp 4: R(X) of T1 and R(X) of T2 are non-conflicting operations because none of them is write operation.

Exp 5: W(X) of T1 and R(X) of T1 are non-conflicting operations because both the operations belong to same transaction T1.

View Serializability

- View Serializability is a process to find out that a given schedule is view serializable or not.

- To check whether a given schedule is view serializable, we need to check whether the given schedule is View Equivalent to its serial schedule.

- A schedule will view serializable if it is view equivalent to a serial schedule.

View Serializability

View Equivalent

- Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:

1. Initial Read: An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

2. Update Read: If in schedule S1, the transaction T1 is reading a data item updated by T2 then in schedule S2, T1 should read the value after the write operation of T2 on same data item.

3. Final write: A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

View Serializability

For Example:

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

Recoverability of Schedule

- Sometimes a transaction may not execute completely due to a software issue, system crash or hardware failure.
- In that case, the failed transaction has to be rollback.
- But some other transaction may also have used value produced by the failed transaction.
- So we also have to rollback those transactions.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-file-organization/ last accessed on 18 September' 2022.


Thank
you

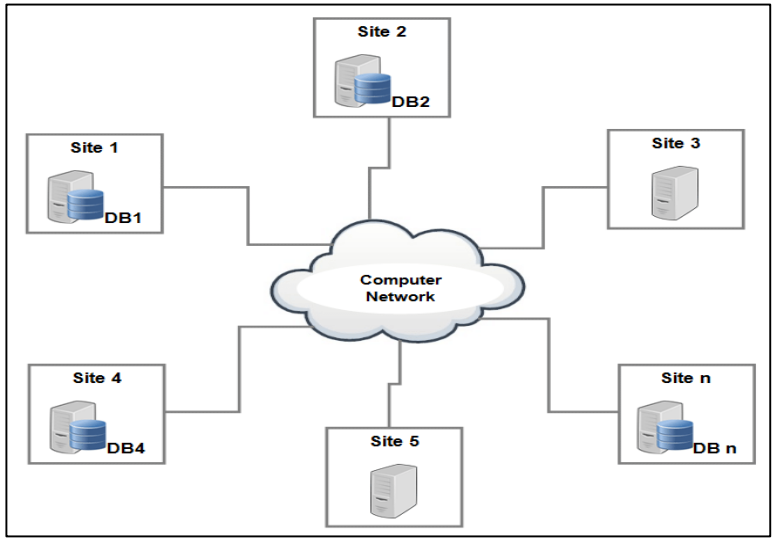
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC – 1003)

Topic: Distributed Database



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- Distributed DBMS
- Types of Distributed Databases
- Data Replication & Fragmentation

Distributed DBMS

- A distributed database is a set of interconnected databases that is distributed over the computer network or internet.

- A Distributed Database Management System (DDBMS) manages the distributed database and provides mechanisms so as to make the databases transparent to the users.

- In these systems, data is intentionally distributed among multiple nodes so that all computing resources of the organization can be optimally used.

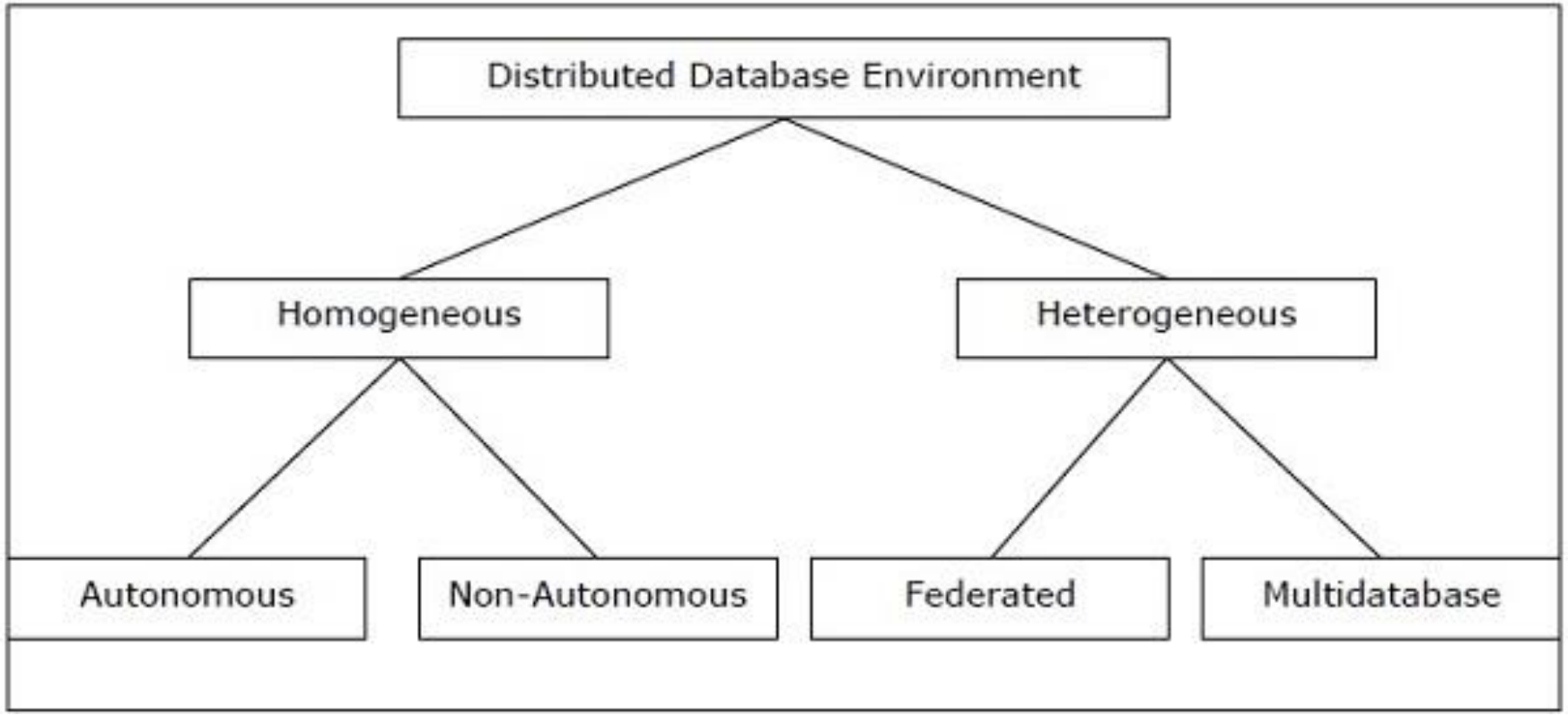
Distributed DBMS

The following factors encourage moving over to DDBMS −

- Distributed Nature of Organizational Units
- Need for Sharing of Data
- Support for Both OLTP and OLAP
- Database Recovery
- Support for Multiple Application Software

Types of Distributed Database

Distributed databases can be broadly classified into homogeneous and heterogeneous distributed database environments.



Types of Distributed Database

Homogeneous Distributed Databases

In a homogeneous distributed database, all the sites use identical DBMS and operating systems.

Types of Homogeneous Distributed Database

There are two types of homogeneous distributed database −

Autonomous: Each database is independent that functions on its own. They are integrated by a controlling application and use message passing to share data updates.

Non-autonomous: Data is distributed across the homogeneous nodes and a central or master DBMS co-ordinates data updates across the sites.

Types of Distributed Database

Heterogeneous Distributed Databases

In a heterogeneous distributed database, different sites have different operating systems.

Types of Heterogeneous Distributed Databases

There are two types of heterogeneous distributed database −

Federated: The heterogeneous database systems are independent in nature and integrated together so that they function as a single database system.

Un-federated (Multidatabases): The database systems employ a central coordinating module through which the databases are accessed.

Data Replication & Fragmentation

Data Replication

- Data replication is the process of storing separate copies of the database at two or more sites. It is a popular fault tolerance technique of distributed databases.

Fragmentation

- Fragmentation is the task of dividing a table into a set of smaller tables. The subsets of the table are called fragments. Fragmentation can be of three types: horizontal, vertical, and hybrid (combination of horizontal and vertical).
- Fragmentation should be done in a way so that the original table can be reconstructed from the fragments. This is needed so that the original table can be reconstructed from the fragments whenever required.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-file-organization/ last accessed on 15 October' 2021.


Thank
you

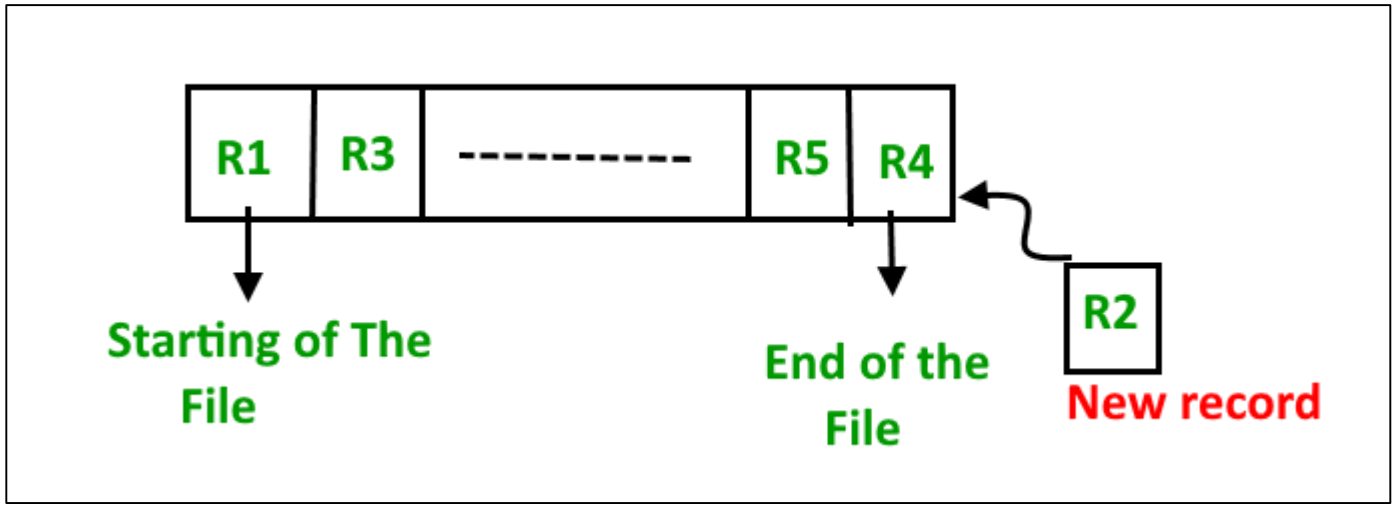
GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)

Accredited with A Grade by NAAC

12-B Status from UGC

Database Management System (BCSC – 1003)

Topic: File Organization



Dr. Nikhil Govil

Assistant Professor, Dept. of CEA, GLA University, Mathura.

Topics to be covered

- File Organization
- Objectives of File Organization
- Indexing in DBMS
- Index Structure
- Indexing Methods

File Organization

- The File is a collection of records. Using the primary key, we can access the records.
- File organization is a logical relationship among various records. This method defines how file records are mapped onto disk blocks.
- File organization is used to describe the way in which the records are stored in terms of blocks, and the blocks are placed on the storage medium.
- Files of fixed length records are easier to implement than the files of variable length records.

Objective of File Organization

- It contains an optimal selection of records, i.e., records can be selected as fast as possible.
- To perform insert, delete or update transaction on the records should be quick and easy.
- The duplicate records cannot be induced as a result of insert, update or delete.
- For the minimal cost of storage, records should be stored efficiently.

Indexing in DBMS

- The index is a type of data structure.
- It is used to locate and access the data in a database table quickly.
- Indexing is used to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.

Index Structure

- Indexes can be created using some database columns.

Search key	Data Reference

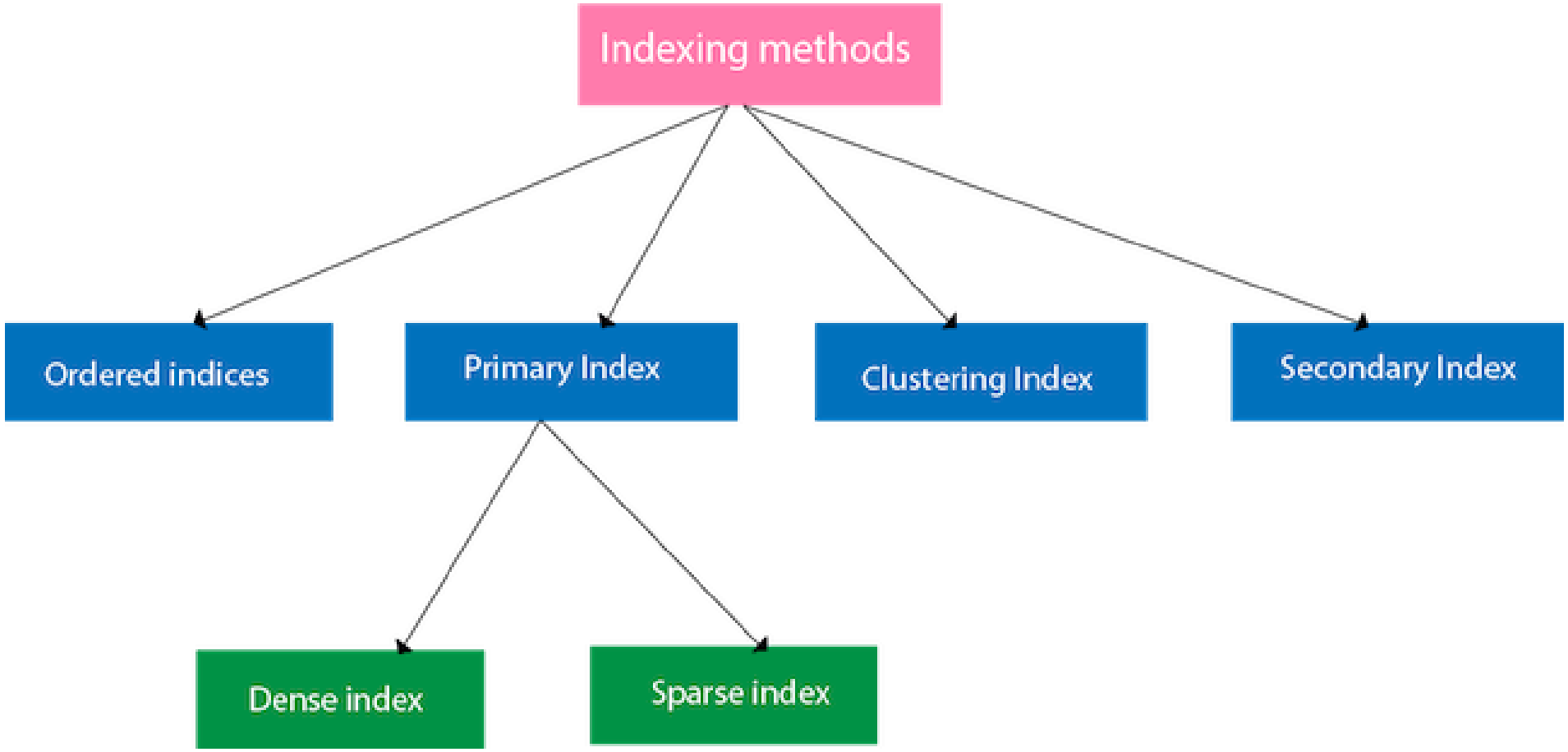
Fig: Structure of Index

- The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.

- The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.

Indexing Methods

- Indexing in DBMS can be performed in following methods:



Order Indices

- The indices are usually sorted to make searching faster. The indices which are sorted are known as ordered indices.

Example:

Suppose we have a student table with thousands of records, each of which is 10 bytes long. Imagine their IDs start from 1 2, 3… and goes on. And we have to search student with ID 678.

- In a normal database with no index, it searches the disk block from the beginning till it reaches 678.
- Now if we want to search record with ID 678, then it will search using indexes.

Primary Index

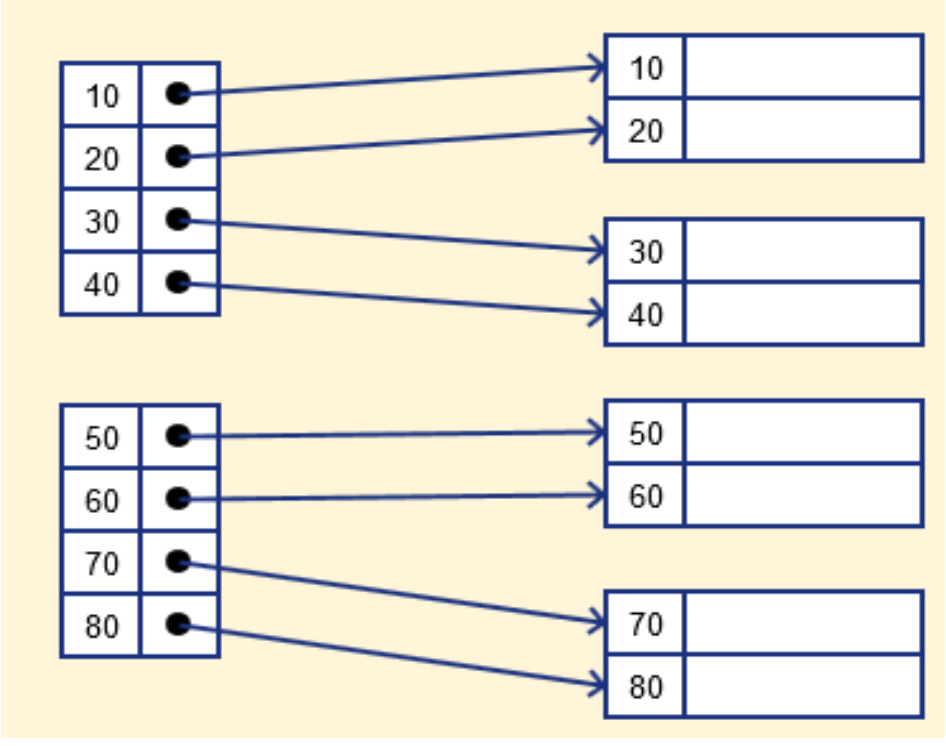
- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.

- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.

- The primary index can be classified into two types as:
  - ❖Dense index and
  - ❖Sparse index.

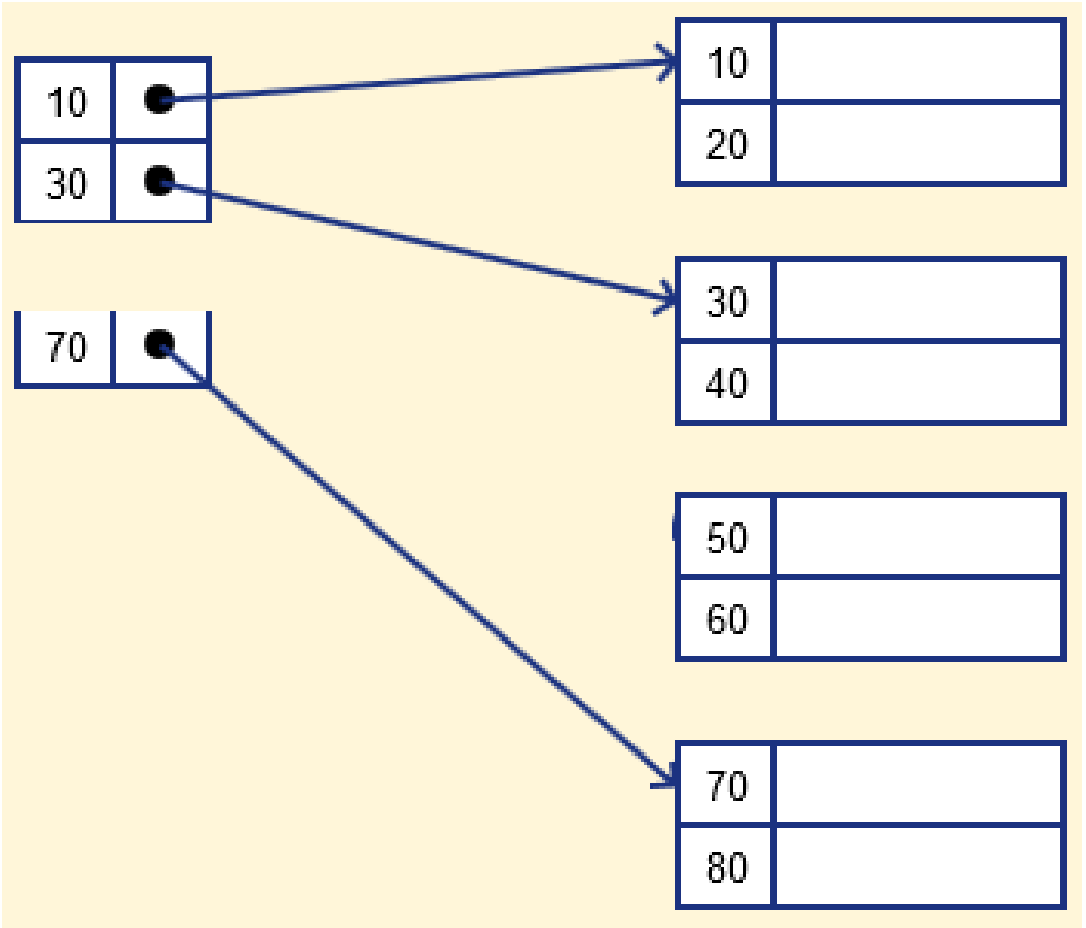
Dense Index

- In dense index, a record is created for every search key valued in the database.
- It helps us to search faster but needs more space to store index records.
- In this Indexing, method records contain search key value and points to the real record on the disk.



Sparse Index

- It is an index record that appears for only some of the values in the file.
- It needs less space, less maintenance overhead for insertion, and deletions.



Clustering Index

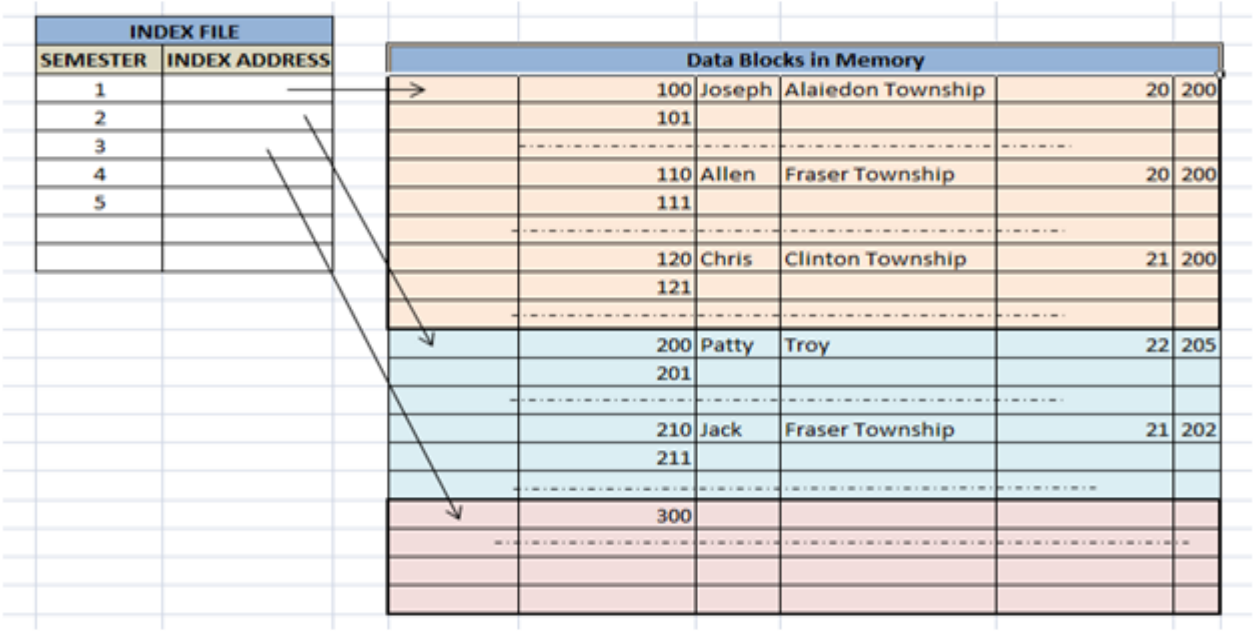
- In some cases, the index is created on non-primary key columns which may not be unique for each record.

- In such cases, in order to identify the records faster, we will group two or more columns together to get the unique values and create index out of them.

- In clustering index, records with similar characteristics are grouped together and indexes are created for these groups.

Clustering Index

- For example, students studying in each semester are grouped together, i.e.; 1st semester students, 2nd semester students, 3rd semester students etc. are grouped.



INDEX FILE	
SEMESTER	INDEX ADDRESS
1	
2	
3	
4	
5	

Data Blocks in Memory

	100	Joseph	Alaiedon Township	20	200
	101				
	110	Allen	Fraser Township	20	200
	111				
	120	Chris	Clinton Township	21	200
	121				
	200	Patty	Troy	22	205
	201				
	210	Jack	Fraser Township	21	202
	211				
	300				

- Here, indexes are created for each semester in the index file. In the data block, the students of each semester are grouped together to form the cluster. The address in the index file points to the beginning of each cluster. In the data blocks, requested student ID is then search in sequentially.

Secondary Index

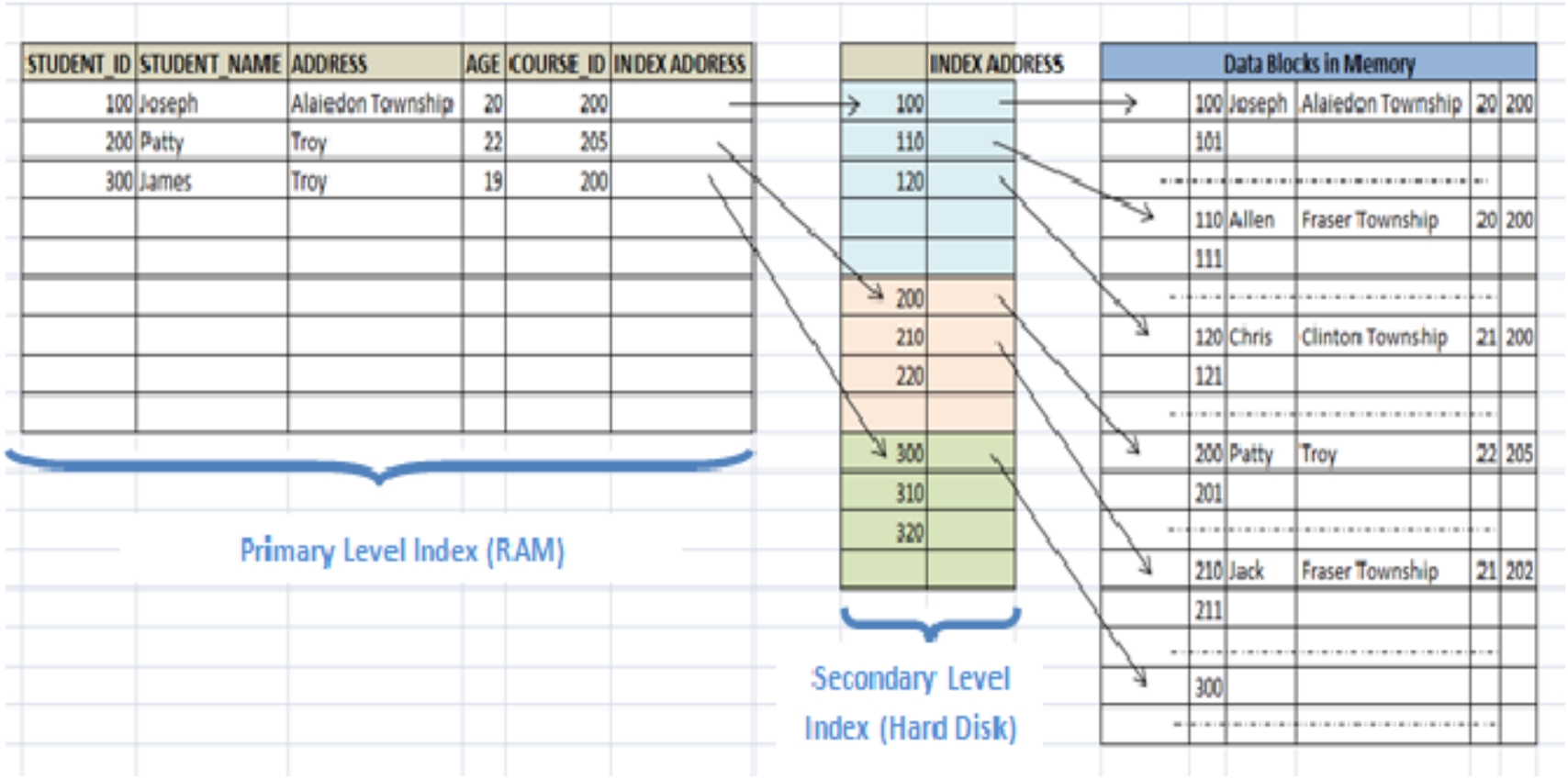
- In the sparse indexing, as the table size grows, the (index, address) mapping file size also grows.

- In the memory, usually these mappings are kept in the primary memory so that address fetch should be faster. And latter the actual data is searched from the secondary memory based on the address got from mapping.

- If the size of this mapping grows, fetching the address itself becomes slower. Hence sparse index will not be efficient.

- In order to overcome this problem next version of sparse indexing is introduced i.e.; Secondary Indexing.

Secondary Index



- In the above diagram, we can see that columns are divided into groups of 100s first. In the secondary memory, these groups are further divided into sub-groups. Actual data records are then stored in the physical memory.

References

- Korth, Silbertz and Sudarshan (1998), “Database Concepts”, 4th Edition, TMH.
- Elmasri and Navathe (2010), “Fundamentals of Database Systems”, 5th Edition, Addision Wesley.
- Date C J,” An Introduction to Database Systems”, 8th Edition, Addision Wesley.
- M. Tamer Oezsu, Patrick Valduriez (2011). “Principles of Distributed Database Systems”, 2nd Edition, Prentice Hall.
- https://www.javatpoint.com/dbms-file-organization/ last accessed on 25 September' 2022.


Thank
you